

DeepSeek review of μ OS++ IIIe

Kernel, ports, platforms and tests

DeepSeek review of μ OS++ Ille — kernel, ports, platforms and tests

- Executive summary
- Second pass code review

μ OS++ Ille SMP kernel — second-pass code review

- Scope
- Summary
- New/updated findings
- Agreement with first pass
- Good practices

μ OS++ Ille AArch32 / AArch64 architecture-port review — second pass

- Scope
- Summary
- New/updated findings
- AArch32 vs AArch64 divergences
- Agreement with first pass
- Good practices

Second-pass review: μ OS++ Ille Cortex-M & POSIX ports

- Scope
- Summary
- New/updated findings
- Agreement with first pass
- Good practices

Second-pass review — μ OS++ Ille test harness ([micro-os-plus-iii-smp.git/tests/](https://github.com/micro-os-plus-iii-smp.git/tests/))

- Scope
- Summary
- New/updated findings
- Agreement with first pass
- Documentation drift
- Good practices

First pass — detailed findings

μ OS++ Ille SMP kernel — read-only code review

- Scope
- Summary
- Findings
- Good practices

μ OS++ Ille AArch32 / AArch64 port review

- Scope
- Summary

- Findings
- AArch32 vs AArch64 divergences
- Good practices

Code review — μ OS++ Ille Cortex-M and POSIX architecture ports

- Scope
- Summary
- Findings
- Good practices

Read-only code review — μ OS++ Ille test harness / platforms / build system

- Scope
- Summary
- Findings
- Documentation drift
- Good practices

DeepSeek review of μ OS++ Ille — kernel, ports, platforms and tests

A read-only review of the three working copies that make up the μ OS++ Ille test workspace: the **SMP kernel** and its harness (`micro-os-plus-iii-smp.git`), the **architecture ports** (`micro-os-plus-iii-{aarch32,aarch64,cortexm,posix-arch}.git`) and the **test platforms** (`tests/platforms/*` , `tests/sources/*` , `test_smpl/`). Nothing in the reviewed trees was modified; this document and its PDF are the only artefacts.

The review was split into four independent passes — kernel, AArch32/AArch64 ports, Cortex-M/POSIX ports, and the test harness/build system — each reading the source and reporting findings with `file:line` evidence and a severity (Critical / High / Medium / Low / Nit). Each pass is reproduced below under its own part.

Executive summary

The design is deliberate and, in the SMP-sensitive core, well documented: the cross-core lock, the deferred publish/claim protocol, the secondary-core release and the semihosting verdict path are coherent and largely correct. The findings below cluster into four themes.

1. Missing synchronisation on shared scheduler state (kernel, High). `join()` , `kill()` and the idle reaper read another core's `current_thread_[]` , thread `state_` and `context_.port_.stack_ptr` without atomics/acquire, while the ports write `stack_ptr` with release stores — a formal C++ data race (`src/rtos/os-thread.cpp:1104,1460` ; `src/rtos/os-idle.cpp:91`). The semihosting file-descriptor table is likewise unsynchronised.

2. Cortex-M SMP: divergence between the two near-duplicate cores (Critical). `switch_stacks()` 's "no ready thread" path spins forever **while holding the kernel spinlock**, freezing the other core (`os-core-m33.cpp:471` , `os-core-rp2350.cpp:448`); the RP2350 copy lost the M33's PRIMASK save/restore and calls `port_put_lock(0)` (`os-core-rp2350.cpp:352`); and neither masks local interrupts around the thread picker, unlike upstream `os-core.cpp:842-869` . The two cores are ~91% duplicated, which is how the drift arose.

3. Memory and time (kernel/ports, High). `malloc` / `new` memory resources discard the requested alignment and no over-aligned `operator new` is defined (`include/cmsis-plus/estd/malloc.h:242,279`); `block_pool::internal_construct_` has an inverted `std::align` failure assert and then overstates the arena (`src/memory/block-pool.cpp:167-179`). On the Pi the AArch32 `clock_highres` reads the raw, admittedly-wrong `CNTFRQ` instead of the calibrated frequency — high-res durations are ~19× off (`aarch32 .../rtos/os-core.cpp` `os-inlines.h:309,315`); the AArch64 MMU leaves TTBR1 walks enabled with `T1SZ=0` (`aarch64 .../mmu.cpp:139`).

4. Test-harness wiring (tests, High/Medium). The Luckfox Lyra harness never calls `smp_install_boot_threads()` / `start_secondary_cores()` , so its suites silently run on core 0 only

(`tests/platforms/aarch32-luckfox-lyra/src/platform-support.cpp:94`); the four Pi platforms define `micro-os-plus::platform` / `platform-support` targets nothing links, and their base interface hard-wires `QEMU_BUILD` and the QEMU linker script (`tests/platforms/aarch32-rpi3b/cmake/platform-library.cmake`); the pinned-toolchain guard exists only in the AArch32/AArch64 definitions.

Also notable. The DWC2 USB gadget stack enumerates only when compiled at `-O0` (a single `-O2` transformation breaks the EP0 path — worked around in `cortexm .../tests.cmake:162`); `SYS_EXIT` 's parameter shape differs between the AArch32 (reason-by-value) and AArch64 (two-word block) ports; secondary-core release timeouts are silent; and the POSIX port's macOS tick is process-directed rather than per-CPU.

Each part below gives the full findings, with confirmed defects separated from suspicions and, where relevant, a note on good practices worth keeping.

Second pass code review

A second, independent pass with fresh eyes over the same four areas. It both re-derived the first pass's conclusions and found items the first pass missed; where the tree had moved on, it says so. Several Cortex-M findings were already fixed by the time of this pass (commit `18d91f1`), and two first-pass kernel items no longer reproduce against the current tree (the POSIX fd-manager null-deref and the `block_pool` "inverted assert").

New conclusions, by area:

- **Kernel.** The POSIX file-descriptor table and FatFs' deferred file/directory lists are mutated from several cores with **no lock at all** (`open` / `close` / `opendir`); `mutex::boosted_prio_` keeps only one waiter's priority, so unlocking an unrelated mutex can drop a still-needed priority inheritance (unbounded inversion); the `BLKSSZGET` / `BLKPBSZGET` / `BLKGETSIZE64` ioctls have **inverted** success tests; allocator `align_size` overflows `size_t` before allocating; user timer callbacks run while the global SMP kernel spinlock is held, and no generic cross-core reschedule IPI exists (`port_smp_ipi` is used only by `kill`); `high_resolution_clock::now()` overflows its 64-bit `cycles*1e9` after ~13 min at 24 MHz.
- **Ports.** The AArch64 fatal-exception reporter still prints through the semihosting-mirrored UART — the very re-entry trap the AArch32 `FaultConsole` avoids; the Luckfox USB stack's "no-op cache maintenance + arbitrary caller buffers" contract is the likely mechanism behind the `-O0`-only enumeration; the DWC2 driver hard-codes a 512-byte bulk MPS; AArch64 still trusts raw `CNTFRQ_EL0` (the same class as the AArch32 `clock_highres` bug); the shared AArch64 `startup.S` hard-codes RAM bounds; a missing `ISB` after `VBAR` .
- **Cortex-M / POSIX.** The first pass's four defects are confirmed and were fixed by commit `18d91f1` ; the scheduler is now ~88% triplicated across `os-core{, -m33, -rp2350}.cpp` with concrete drift (`|1` vs `&~1` , a 3- vs 2-field `_smp_klock`); `hw_result::semi_write0` 's inline asm omits the `r2/r3/ip/lr` clobbers the kernel's own `call_host` declares; on macOS the tick can stall the clock entirely (no `timer_create`); the per-board `hw.sh` collapses OpenOCD death/error into a 600 s timeout.

- **Tests / build.** `test_smp/run-qemu.sh` and `run-host.sh` exit 0 when the image named by `UOS_QEMU_ONLY` / `UOS_RUN_ONLY` is missing (a renamed image becomes a permanently green case); the duplicate-source gate exempts all of `tests/`; the live Luckfox `platform-support.cpp` is the stale copy (no 3 s secondary handshake, no console mirror); hardware-only `xpm run test` runs zero tests; `pico2-1cpu` / `2xcortex-m33` carry no CTest labels.

The four second-pass reports follow.

µOS++ IIIe SMP kernel — second-pass code review

Scope

Read-only. No source file was modified, created or deleted; no build was run. Reviewed under `/home/dan/Work`:

- Kernel sources `micro-os-plus-iii-smp.git/src/` (rtos, memory, libc, libcpp, posix-io, semihosting, startup, utils) and headers `micro-os-plus-iii-smp.git/include/cmsis-plus/`.
- Harness `micro-os-plus-iii-smp.git/tests/sources/` and `micro-os-plus-iii-smp.git/test_smp/*.sh`.
- Because the prompt names `lock_state[]`, `_port_ctx_pending[]`, deferred publish/claim and the ready-list picker, and those live in the SMP ports, the port half of the kernel was also read where needed: `micro-os-plus-iii-aarch32.git/src/rtos/os-core.cpp`, `.../include/cmsis-plus/rtos/port/os-inlines.h`, `.../test/boards/*/src/rtos/port_isr.cpp`, `.../src/context_switch.cpp`, and the host mirror `micro-os-plus-iii-posix-arch.git/src/host_cpu.cpp` + `src/rtos/os-core.cpp`. Findings there are labelled as port scope.

This pass deliberately tries to *not* repeat `/tmp/opencode/review-kernel.md`. Where it confirms, disputes or sharpens a first-pass item, it says so.

Summary

The single-core kernel logic is mature; the remaining real defects cluster in (a) code that assumes an external lock that the caller does not actually hold on SMP, and (b) arithmetic/state that was never widened for SMP or 64-bit time. The most serious new items are: the **POSIX fd table and the FatFs deferred file/directory lists are mutated from multiple cores with no lock at all** (`open` / `close` / `opendir`), the **priority-inheritance bookkeeping stores only one waiter's priority per mutex** (raising inversion), the **BLK* ioctls have inverted success tests**, and the **allocator size arithmetic overflows** `size_t`. On the ports, the biggest new item is that a **user timer callback runs while the global SMP kernel spinlock is held**, and that a **thread woken on another CPU's affinity is not nudged by an IPI** (no generic reschedule IPI is ever sent; `port_smp_ipi` is only used in `kill()`).

Severity tally: 0 Critical, 6 High, 10 Medium, 6 Low/Nit (22 findings).

Things I could not verify statically (treat as unconfirmed): the exact runtime effect of the `st_mode / __posix_stat` path (I did not locate a live `stat` implementation to read end-to-end); the `millisec * 1000u` call sites (accepted from pass 1 without re-deriving every one); and the Luckfox/RK3506 `port_smp_ipi` implementation (I read only the Raspberry Pi board copy).

New/updated findings

[HIGH] FatFs file/directory deferred lists are unsynchronised — include/cmsis-plus/posix-io/file-system.h:822-831, 845-987; src/posix-io/file.cpp:76

```
// file-system.h
inline void
file_system::add_deferred_file (file* fil)
{
    deferred_files_list_.link (*fil);          // no lock
}
...
if (deferred_files_list_.empty ())
    fil = new file_type (*this);
else
    fil = static_cast<file_type*> (deferred_files_list_.unlink_head ());
```

and from the hot path:

```
// file.cpp:74-76 (file::close)
get_file_system ().add_deferred_file (this); // no lock
```

CONFIRMED (static). `allocate_file` / `allocate_directory` / `deallocate_*` plus `add_deferred_*` run on a per-file-system intrusive list with no critical section, no per-FS lock and no atomics. The `file_lockable<>` wrappers hold `impl_instance_.locker()` for `read` / `write` / `lseek`, but `close()` in `file::close()` links the object onto the deferred list *outside* any lock, and `allocate_file` is reached from the open path. On SMP, two cores doing `open()` / `close()` / `opendir()` at once interleave `unlink_head()` / `link()` and corrupt the list; a re-used object can be destructed twice (`delete f` on a node another core just reused) or leaked. This is a real cross-core race and is not in the first-pass report.

[HIGH] POSIX file-descriptor table is unsynchronised — src/posix-io/file-descriptors-manager.cpp:118-205

```
descriptors_array__[i] = io; // allocate(): plain scan+write
io->file_descriptor (static_cast<int> (i));
...
descriptors_array__[fildes]->clear_file_descriptor (); // deallocate()
descriptors_array__[fildes] = nullptr;
```

CONFIRMED (static). `descriptors_array__` / `size__` are process-wide statics and none of `allocate`, `assign`, `deallocate`, `valid`, `io`, `socket` takes the kernel lock or an atomic. Two cores scanning for a free slot can both pick the same index (one overwrites the other, one `io` is left with no fd), or one can `deallocate()` a slot another core is concurrently installing. `io::close()` (`src/posix-io/io.cpp:207`) calls `deallocate` with no outer lock. Same class of bug as the deferred lists above.

Note (sharper than pass 1): pass 1 reported a null-deref in `deallocate()` / `socket()` / `valid()`. In the tree as checked out those checks are present — `deallocate` returns `EBADF` on a null slot (`file-descriptors-manager.cpp:189-195`), `socket()` tests `io == nullptr` (line 219), and `valid()` tests the slot (lines 109-111). I therefore **disagree** with the null-deref finding as stated; the surviving defect is the missing synchronisation.

[HIGH] `mutex` keeps only one waiter's priority: lost priority inheritance — src/rtos/os-mutex.cpp:779-780, 900-934

```
if (protocol_ == protocol::inherit)
{
    thread::priority_t prio = th->priority ();
    boosted_prio_ = prio; // <-- overwritten by each new waiter
    ...
    if ((owner != nullptr) && (boosted_prio_ > owner->priority_inherited ()))
        owner->priority_inherited (boosted_prio_);
}
```

and on unlock, when owned mutexes remain:

```
thread::priority_t max_prio = 0;
for (auto&& mx : *thread_mutexes)
    if (mx.boosted_prio_ > max_prio) max_prio = mx.boosted_prio_;
boosted_prio_ = max_prio;
owner_->priority_inherited (boosted_prio_);
```

CONFIRMED (static). `boosted_prio_` is a single scalar per mutex, not the maximum over waiters. Scenario: owner holds `M1` and `M2`. Waiter A (prio 10) blocks on `M1` → `M1.boosted_prio_ = 10`, owner inherits 10. Waiter B (prio 5) blocks on `M1` → `M1.boosted_prio_ = 5` (the 5 replaces the 10; the boost *test* correctly refuses to lower the owner, but the stored value is lost). The owner then unlocks `M2`, whose `boosted_prio_` is non-`none` (e.g. a `protocol::protect` ceiling or another waiter), so the recompute walks the owner's mutexes, sees `M1.boosted_prio_ == 5`, and sets the owner's inherited priority to **5** while A (prio 10) is still blocked on `M1`. The owner can then be preempted by priority-6..9 work → classic, unbounded priority inversion. The fix is to store the per-mutex waiter maximum (`boosted_prio_ = max(boosted_prio_, prio)`). Not in the first-pass report.

**[HIGH] Allocator size arithmetic overflows `size_t` before allocating — include/
cmsis-plus/rtos/os-memory.h:84-88; src/memory/first-fit-top.cpp:126-132; src/
memory/lifo.cpp:73-79**

```
// os-memory.h
constexpr std::size_t
align_size (std::size_t size, std::size_t align) noexcept
{
    return ((size) + (align)-1L) & ~((align)-1L);    // wraps for size > SIZE_MAX-align
}

// first-fit-top.cpp
std::size_t alloc_size = rtos::memory::align_size (bytes, chunk_align);
alloc_size += block_padding;
alloc_size += chunk_offset;
```

CONFIRMED (static). A caller passing a near-`SIZE_MAX` `bytes` (the public `memory_resource::allocate(bytes, align)` interface, `pmr`, or `new char[huge]`) wraps `align_size` to a tiny value. `do_allocate` then considers the request "small", may `std::align` with the original huge `bytes` (which fails) and, in release builds (`assert` compiled out), returns a payload for a request that cannot fit → heap corruption. The first pass caught the *signed*→*unsigned* `static_cast<int>` fit test (first-fit-top.cpp:143 / lifo.cpp:91) but not this upstream overflow. Sharper: guard with `if (bytes > total_bytes_ - overhead) return nullptr;`.

[HIGH] User timer callbacks run with the global SMP kernel spinlock held — src/rtos/internal/os-lists.cpp:489-530 → src/rtos/os-timer.cpp:368-394

```
// clock_timestamps_list::check_timestamp
for (;;)
{
    interrupts::critical_section ics;    // takes _smp_klock on every SMP port
    ...
    if (now >= head_ts)
        const_cast<timestamp_node*> (head ())->action ();    // -> timer::internal_callback
    ...
}
...
// timer::internal_callback
func_ (func_args);    // user function, still under the lock
```

CONFIRMED (static). On the aarch32/posix ports `interrupts::critical_section` takes the single global `_smp_klock` (`aarch32/.../os-inlines.h:222-238`, `posix-arch/.../os-core.cpp`); `check_timestamp` holds it across `action()` → `timer_node::action` → `timer::internal_callback`, which calls the user callback. So a slow (or blocking) timer callback stalls **every other core** on the kernel spinlock, and a callback that tries to block would deadlock (the lock is held). This is inherent to running the callback inside the list lock; on SMP the callback should be deferred out of the critical section.

[HIGH] BLK ioctls have inverted success tests — src/posix-io/block-device.cpp:156, 174, 192

```
case BLKSSZGET:
{
    std::size_t* sz = va_arg (args, std::size_t*);
    if (sz == nullptr || impl ().block_logical_size_bytes_ != 0)
        { errno = EINVAL; return -1; }    // errors when the size IS known
    *sz = impl ().block_logical_size_bytes_;    // returns 0 when it is unknown
    return 0;
}
```

`BLKPBSZGET` (line 174) and `BLKGETSIZE64` (line 192, `num_blocks_ != 0`) are identical.

CONFIRMED (static). The guard should be `== 0` (unknown). As written the ioctl fails (`EINVAL`) exactly when it has a value to report, and "succeeds" by handing the caller a zero. Every caller that queries the sector size / device size gets `EINVAL` on a healthy device. Not in the first-pass report.

[MEDIUM] Threads are never nudged across cores: no generic reschedule IPI — src/rtos/os-thread.cpp:721; port reschedule has no peer wake; `port_smp_ipi` used only at os-thread.cpp:1497

```
// resume(), after linking to the ready list:
port::scheduler::reschedule ();           // only affects the CURRENT core
```

The only `port_smp_ipi()` call in the kernel is in `kill()`.

CONFIRMED (static). When thread T becomes ready with `cpu_affinity` that excludes the core that woke it, the waker's `reschedule()` cannot dispatch T and does **not** signal the eligible core. T waits until that core's own next tick (1 ms default) or any other interrupt. Correctness is preserved (bounded by the tick), but this is a real scheduling-latency bug and contradicts the comment in `kill()` about "actively trigger reschedule IPI" as a general mechanism. On a tickless/idle-sleep configuration it would be unbounded. (SUSPECTED to matter for real-time tests only; I did not run them.)

[MEDIUM] `clock_highres::now()` is in a different unit/domain than its sleeps — src/rtos/os-clocks.cpp:793-809 vs 191-291; port aarch32 `.../os-inlines.h:300-327`

```
clock::timestamp_t
clock_highres::now (void)
{
    if constexpr (port::clock_highres::has_hardware_counter ())
        return port::clock_highres::hardware_counter (); // free-running timer cycles
    ...
}
```

But `clock::sleep_for / sleep_until / internal_wait_until_` (lines 191-291, 395-439) use `steady_now() / steady_count_` (one count per 1 ms tick) and the `steady_list_`. On the aarch32 port `has_hardware_counter()` is `true` (`timer_arm::get_count()`), but `clock_highres::internal_increment_count()` still ticks `steady_count_` once per SysTick.

CONFIRMED (static, API-level). `hrclock.now()` returns raw timer counts while `hrclock.sleep_for(d)` interprets `d` as ticks and `sleep_until(hrclock.now()+d)` mixes the two domains. The first pass noted a related condvar/clock issue; this is the sharper statement that the high-res clock's `now()` and its sleep API do not share a time base. (`hrclock` is a `clock`, not `adjustable_clock`, so it overrides `now()` but not `sleep_*`.)

[MEDIUM] `high_resolution_clock::now()` overflows 64-bit multiply after ~13 min — `src/libcpp/chrono.cpp:119-124`

```
return time_point{
    duration{
        duration{ cycles * 1000000000ULL
            / rtos::hrclock.input_clock_frequency_hz () }
        + realtime_clock::startup_time_point.time_since_epoch () } //
};
```

CONFIRMED (static). `cycles` is `hrclock.now()` (uint64 raw counter). On the aarch32 port `input_clock_frequency_hz()` is `timer_arm::frequency()` (e.g. 24 MHz), so `cycles * 1e9` wraps once `cycles > 2^64/1e9 ≈ 1.84e10`, i.e. after ≈ 768 s ≈ 12.8 minutes of the timer running. `high_resolution_clock::now()` then returns a wildly wrong point (not monotone). The comment acknowledges the range trade-off but the overflow is real. Compute as `cycles / freq * 1e9 + cycles % freq * 1e9 / freq` or use 128-bit.

[MEDIUM] `atexit()` bounds check is outside the lock — `src/libc/stdlib/atexit.cpp:118-133`

```
assert (__atexit_count < OS_INTEGER_ATEXIT_ARRAY_SIZE); // before the lock
#ifdef NDEBBUG
    if ((type != __et_atexit) || (__atexit_count >= OS_INTEGER_ATEXIT_ARRAY_SIZE))
        return -1;
#endif
os::rtos::scheduler::critical_section scs; // lock only now
__atexit_functions[__atexit_count++] = fn;
```

CONFIRMED (static, narrow). With `NDEBBUG` and the array one short of full, two cores can both pass the unlocked check, then serialize on the lock: the second writes `__atexit_functions[OS_INTEGER_ATEXIT_ARRAY_SIZE]` — one past the end (and `__atexit_count` overshoots). The check must be inside the critical section. Strictly single-threaded use is unaffected.

[MEDIUM] `block_pool::do_allocate` ignores the requested alignment — `src/memory/block-pool.cpp:48-82, 134-138`

```
void* block_pool::do_allocate (std::size_t bytes, std::size_t alignment)
{
    assert (bytes <= block_size_bytes_);
    if (first_ == nullptr) return nullptr;
    void* p = static_cast<void*> (first_);    // alignment discarded
    ...
}
```

Also `do_max_size()` returns `block_size_bytes_ * blocks_` (line 137), which can overflow `size_t`.

CONFIRMED (static). Blocks are only `alignof(void*)`-aligned; an over-aligned request is silently under-served (same class as pass 1's `malloc` / `new` alignment item, but a *different* allocator, so pass 1 did not cover it). Worse than `malloc`, this one never even sees `max_align`. `do_max_size`'s multiply wants a saturation/checked multiply.

[MEDIUM] Periodic software timer can fire repeatedly inside one tick — `src/rtos/os-timer.cpp:373-379 + src/rtos/internal/os-lists.cpp:500-529`

```
if (type_ == run::periodic)
{
    timer_node_.timestamp += period_;    // may stay in the past
    clock_>steady_list ().link (timer_node_);
}
```

`check_timestamp` loops `while (now >= head_ts)`, and the callback runs inside that loop.

CONFIRMED (static). If the callback duration exceeds the period (or the ISR was delayed), the re-armed timestamp is still `<= now`, so the same timer fires again on the next loop iteration — a burst of callbacks in one tick, and, because each iteration re-enters the critical section while the callback runs, a potentially very long ISR. Common embedded pitfall; not in pass 1.

[MEDIUM] `is_thread_allowed_on_cpu()` pins any user thread named `idleN` — `src/rtos/os-core.cpp:493-517`

```
const char* name = th->name ();
if (name != nullptr && name[0] == 'i' && name[1] == 'd' && name[2] == 'l'
    && name[3] == 'e')
{
    if (name[4] == '\0' || name[4] == '0') return (cpu == 0);
    if (name[4] >= '1' && name[4] <= '9' && name[5] == '\0')
        return (cpu == static_cast<unsigned> (name[4] - '0'));
}
return (th->cpu_affinity () & (1u << cpu)) != 0;
```

CONFIRMED (static). The name rule cannot distinguish the boot-window idle threads from an application thread the user names `idle1`. If the user's thread is created with `OS_NCPU == 4`, it is pinned to core 1 forever regardless of its requested affinity (and a thread named `idle5` is pinned to a non-existent core and never runs). The identity check against `os_idle_thread_core[]` is sound; the name heuristic is a foot-gun that should at least require the thread to also be on the top-level/first-run list, or be removed once registration is complete.

[MEDIUM] Idle reaper stops at the first still-live head — `src/rtos/os-idle.cpp:94-103`

```
if (live)
{
    break;                // gives up on ALL later terminated threads
}
```

CONFIRMED (static). The terminated list is FIFO; if the head is still live on another core (the common case right after `internal_exit_`), the reaper `break`s and never reaps any other dead thread behind it until the head finally dies. With a busy core 0 (which is the only core running `os_rtos_idle_actions`) destroyed threads can accumulate. A `continue`-style scan (skip live, reap others) would be better, or unlink the live head's successor. Not in pass 1.

[MEDIUM] Termination can tear down a thread whose timeout node is still on a stack frame — `src/rtos/os-thread.cpp:1533-1544` vs `src/rtos/internal/os-lists.cpp:344-356`

```
// kill()
if (waiting_node_ != nullptr) waiting_node_->unlink ();
if (clock_node_ != nullptr) clock_node_->unlink ();
```

```
// timeout_thread_node::action (timer ISR)
this->unlink ();
... th->resume ();
```

SUSPECTED. `kill()` unlinks `waiting_node_ / clock_node_` under an `interrupts::critical_section`, but the target may already have been woken by the timer ISR and be *between* `internal_unlink_node()` returning and the node going out of scope. The SMP `kill()` gate confirms the thread is off every CPU before claiming, so the node should already be unlinked by then; I could not construct a concrete window statically, hence SUSPECTED rather than CONFIRMED. Worth a comment/assert documenting the invariant.

[LOW] `block_pool::internal_reset` writes the last link word past the arena — `src/memory/block-pool.cpp:194-210`

```
char* p = static_cast<char*> (pool_addr_); // may be advanced by std::align
for (std::size_t i = 1; i < blocks_; ++i) { ... p = pn; }
*(static_cast<void**> (static_cast<void*> (p))) = nullptr;
```

CONFIRMED (static). `internal_construct` advances `pool_addr_` by up to `alignof(void*)-1` while `total_bytes_ = blocks_ * block_size_bytes_` is *not* reduced. The block chain therefore starts at the advanced address and its end can exceed the caller's buffer by the alignment slack (and `free_bytes_` overstates). This is the surviving half of pass 1's block-pool item (see Agreement). `blocks_ == 0` also still writes one `nullptr` into a zero-length arena.

[LOW] `first_fit_top::do_deallocate` "already freed" branch trusts a reused header — `src/memory/first-fit-top.cpp:277-282, 387-403`

```
else if (reinterpret_cast<char*> (prev_chunk) + prev_chunk->size
        > reinterpret_cast<char*> (chunk))
{
    // Already freed.
    allocated_bytes_ += chunk->size;    // chunk->size is stale/overwritten
    free_bytes_ -= chunk->size;
    ++allocated_chunks_; --free_chunks_;
```

CONFIRMED (static). Once a chunk has been coalesced into `prev_chunk`, the old header (`chunk->size`) is ordinary payload and may hold arbitrary data; the branch nevertheless uses it to adjust statistics and `++allocated_chunks_`. It is also only reached when `prev_chunk` is a *free* chunk, so a double free whose predecessor is allocated is not detected. Debug-only heuristic, but misleading.

[LOW] `__posix_getcwd` does not handle `size < 5` or `buf == nullptr` — src/semihosting/c-syscalls-semihosting.cpp:671-677

```
char*
__posix_getcwd (char* buf, size_t size)
{
    strncpy (buf, "/tmp", size);
    return buf;
}
```

CONFIRMED (static). Not NUL-terminated for `size < 5`; `strncpy(..., 0)` writes nothing; `buf == nullptr` crashes. POSIX says `NULL` + `ERANGE` for a short buffer and permits `getcwd(NULL, 0)` to allocate (this faults). (Pass 1 also flagged this; retained here only because my wording adds the `NULL` case.)

[LOW] `memory_resource::allocate/deallocate` counters are non-atomic — include/cmsis-plus/rtos/os-memory.h:1292, 1315

```
++allocations_;
return do_allocate (bytes, alignment);
```

CONFIRMED (static). `allocations_` / `deallocations_` are plain `size_t`; `operator new` / `malloc` wrap calls in a lock, but the public `memory_resource` interface does not, so direct concurrent use loses counts (statistics only; no corruption).

[LOW] Block-device bounds arithmetic can overflow `size_t` — src/posix-io/block-device.cpp:76, 109; block-device-partition.cpp:149, 163

```
if (blknum + nblocks > impl ().num_blocks_) { errno = EINVAL; return -1; }
...
return parent_.read_block (buf, blknum + partition_offset_blocks_, nblocks);
```

CONFIRMED (static). A near-`SIZE_MAX` `blknum` / `nblocks` wraps the sum and bypasses the bounds check, reaching the backend with a bogus block range. The callers compute these from `offset_`, so exploitation needs a large `lseek`, but the checks should compare `blknum > num_blocks_ - nblocks` (after validating `nblocks <= num_blocks_`).

[LOW] `run-hw.sh` treats any `^Error:` line as fatal and races its `tee` — `test_smpl/`
`run-hw.sh:230, 239-243`

```
"$OPENOCD" "${args[@]}" > >(tee "$log") 2>&1 &
local ocd=$!
...
if grep -qE '^Error: ' "$log" 2>/dev/null; then rc=5; break; fi
```

CONFIRMED (static). OpenOCD emits many non-fatal `Error:` diagnostics (optional flash/scan targets); one turns a passing test into `OPENOCD ERROR`. `kill "$ocd"; wait "$ocd"` does not wait for the `tee` process substitution, so the next test's `rm -f "$log"` can race a still-writing `tee`. (Pass 1 flagged this family; the `tee` race is the sharper part.)

Agreement with first pass

- **AGREE** — **cross-core reads of** `current_thread[]`, `state_`, `stack_ptr` (`os-thread.cpp:1099/1460`, `os-idle.cpp:91`, `os-sched.h:59`). Still formally a data race; the busy-wait gates make it a suspected-rare, not a demonstrated, failure. My pass adds the mechanism: the picker's `state_`/`stack_ptr` reads and `internal_relink_running_`'s writes are serialised by `_smp_klock` on the switch path, so the unsynchronised readers in `join()`/`kill()`/reaper are the only exposure, exactly where pass 1 pointed.
- **AGREE** — `malloc`/`new` **drop the requested alignment** (`malloc.h:242`, `new.cpp` has no `align_val_t` overloads). Independently confirmed; I add that `block_pool::do_allocate` and `memory_resource::allocate` also discard it.
- **AGREE (partially)** — `block_pool` **overstates the arena / under-checks size**. But I **DISAGREE** with the "inverted failure assert" half: the tree as checked out reads `if (res == nullptr) { assert (res != nullptr); }` (`block-pool.cpp:171-174`), i.e. the direction is already correct. What remains is the un-reduced `total_bytes_` and the OOB link write (`internal_reset_`), which I reported above.
- **AGREE** — **semihosting fd table unsynchronised** (`c-syscalls-semihosting.cpp`). My pass also finds the `POSIX` fd table (`file-descriptors-manager.cpp`) and the FatFs deferred lists with the same defect.
- **AGREE** — `size_t → int` **fit-test truncation** (`first-fit-top.cpp:143`, `lifo.cpp:91`). I raise the severity by adding the `align_size` overflow above it.
- **AGREE** — `millisec * 1000u` **overflow** (`os-c-wrapper.cpp`). Accepted; I did not re-derive every call site, but cite `clock_systick::ticks_cast` (`os-clocks.h:885`) as the same overflow shape inside the kernel proper.
- **AGREE (with a sharper cause)** — **condvar ETIMEDOUT from the wall clock** (`os-condvar.cpp:771-811`). The sharper companion is the `clock_highres` domain mismatch above.
- **PARTIALLY VERIFIED** — **invalid** `st_mode` (`__posix_stat`). I accept the `S_IFREG|S_IFCHR` analysis on its face but did not read a live `stat` implementation end-to-end; marked unverified.

- **DISAGREE — POSIX fd-manager null-deref.** `deallocate()` (:189), `socket()` (:219) and `valid()` (:106-116) all null-check in the current tree; the first-pass snippet no longer matches. The real defect is the race (reported above). This looks like a first-pass finding against an older revision.
- **DISAGREE — `block_pool::internal_construct_` inverted assert.** Direction is correct in the current tree (see above).

Good practices

- The deferred publish/claim protocol is coherent and well argued: the claim (`stack_ptr = nullptr`) happens before the lock is dropped, the publish is staged per CPU and applied only after SP has left the outgoing stack (`aarch32 os-core.cpp:288-321`, `context_switch.cpp:62-84`, `posix-arch os-core.cpp:325-358`), and the picker's `th == old_thread || (state_ != running && stack_ptr != nullptr)` guard is the matching read side (`os-core.cpp:600-615`).
 - Kernel-lock release ordering (`owner` / `depth` cleared *before* the lock word, with a `dmb` / `dsb`) is correct and the comment explains the exact deadlock it prevents (`posix-arch os-core.cpp:360-373`, `aarch32 os-core.cpp:302-319`).
 - `reschedule()` correctly refuses to switch while the core still owns the kernel lock and defers via `_port_ctx_pending[]`; the port ISR honours `locked() && owner == cpu` before switching (`port_isr.cpp:132-141`).
 - `internal_relink_running_` re-checks `state_ == running` under the lock, so a terminating thread is not re-queued (`os-thread.h:2428-2452`).
 - `condition_variable::wait` links the waiter before releasing the mutex and the claim protocol prevents the woken-but-still-running thread from being run on a second core in the window (`os-condvar.cpp:589-621`) — a non-obvious SMP point that is actually handled.
 - Per-core idle threads are registered by the harness (`test-smp-boot.cpp:45-59`) and the name/identity rules are documented in `os-core.cpp:481-517`; I verified the secondary-core idle path is the harness's responsibility, not a missing kernel registration.
 - The board contract (`test_smpl/src/board-contract.cpp`) and the `run-hw.sh` one-test-per-power-cycle rule are strong, deliberate guard rails.
-

µOS++ IIIe AArch32 / AArch64 architecture-port review — second pass

Scope

Independent, read-only second pass over the architecture ports and their board trees, run against the **current working tree** (not the tree the first report was written from):

- `/home/dan/Work/micro-os-plus-iii-aarch32.git`: `include/` (`port_ctx.hpp`, `semihosting.hpp`, `timer_arm.hpp`, `exception_handler.hpp`, `mmu.hpp`, `cmsis-plus/rtos/port/{os-c-decls.h,os-inlines.h}`), `src/` (`context_switch.cpp`, `exception_handler.cpp`, `handlers.cpp`, `semihosting-exit.cpp`, `smp_secondary.cpp`, `rtos/os-core.cpp`) and `test/boards/{rpi-zero-2w,luckfox-lyra}/**` including the whole `luckfox-lyra/usb/` DWC2 stack.
- `/home/dan/Work/micro-os-plus-iii-aarch64.git`: same top-level layout plus `test/boards/rpi-zero-2w/**`.

Nothing was modified; this report is the only artifact written. Hardware-only behaviour (CNTFRQ truthfulness, GIC trigger type, spin-table release slots) is marked SUSPECTED where it cannot be proven from the sources.

Summary

The SMP core (recursive lock, deferred publish, claim-by-null, tripwire) is sound and largely shared in spirit. The most important *new* result is that **three of the first pass's top findings are already fixed in the tree** (calibrated `clock_highres`, two-word `SYS_EXIT` block, `TCR_EL1.EPD1`), while the *underlying divergence* that produced the clock bug — AArch64 still trusts raw `CNTFRQ_EL0` — was not propagated.

The sharpest new defects are: the **AArch64 fatal-exception reporter still prints through the semihosting-mirrored UART**, exactly the failure mode the AArch32 `FaultConsole` was written to avoid; a **DMA cache contract that is unenforced for arbitrary caller buffers** (the concrete mechanism behind "enumerates only at `-00`"); **hard-coded 512-byte bulk MPS** in the DWC2 driver; and **hard-coded DRAM bounds** in the shared AArch64 `startup.S` mirroring the anti-pattern AArch32 removed.

New/updated findings

[High] AArch64 fatal-exception dump re-enters the semihosting trap — `include/.../exception_handler.cpp:53-62` + `test/boards/rpi-zero-2w/include/uart.hpp:74-84`

`port_fatal_exception()` dumps through the mirroring console:

```
uart::uart1 << "\n\n!!! FATAL EXCEPTION (core " << static_cast<int>(cpu) << ") !!!\n";
...
while (1) __asm__ volatile("wfi");
```

and `uart.hpp`'s `puts()` mirrors every byte to semihosting when `SEMIHOST` is defined:

```
#if defined(SEMIHOST)
    semihosting::write_str(str);    // HLT #0xF000
#endif
```

This is precisely the hazard the AArch32 port documented at length (`aarch32/src/exception_handler.cpp:15-37`) and worked around with a UART-only `FaultConsole` (`puts_uart / putc_uart`, lines 38-113). On AArch64 the fault path was never hardened: if the debugger is absent, lost, or the `HLT` itself is the thing that broke, the first character of the dump raises another synchronous exception into the same handler. The same mirroring console is used by `test/boards/rpi-zero-2w/src/rtos/port_isr.cpp:33,43` (`validate_context`) and `src/context_switch.cpp:15` (`port_ctx_switch_corrupt`). **CONFIRMED** (code path), severity depends on debugger presence. Fix: give AArch64 the same `puts_uart`-only fault console as AArch32.

[High] The luckfox USB DMA cache contract is unenforced; arbitrary caller buffers are handed to DMA with no-op cache ops — `test/boards/luckfox-lyra/usb/src/usb_env_stateos.cpp:94-96`, `usb/src/usb_vendor_gadget.cpp:485,557`

The RTOS shim makes cache maintenance a no-op on the stated assumption that *every* DMA buffer lives in the Normal-NC `.dma_nc` window:

```
void invalidate_dcache_range(std::uint32_t, std::uint32_t) { }    // no-op
void flush_dcache_range    (std::uint32_t, std::uint32_t) { }    // no-op
```

But the public vendor API does not enforce that assumption:

```
req->buf = const_cast<void*>(buf);    // usb_vendor_write, :485 - caller's buffer
...
req->buf = buf;                        // usb_vendor_read, :557 - caller's buffer
```

A caller that passes ordinary cacheable `.bss` / `.rodata` / stack memory gets DMA to/from physical RAM while the CPU holds stale cache lines (or leaves dirty lines the device never sees). At `-00` the CPU re-reads memory and it works; at `-02` the cached value persists and enumeration/transfer fails — the exact signature the first pass recorded as "DWC2 gadget only enumerates at `-00`". The in-tree gadget's own buffers (`usb_data_buffer`, `ring_buffer`, `bridge_hdr_buf`, `setup_buffer_`) are correctly `.dma_nc`, so the mechanism is **CONFIRMED** for the API, and **SUSPECTED** as the specific `-00` root cause; I

could not find the `-02`-only regression line by inspection alone. Fix: reject/convert non-`.dma_nc` buffers in `queue()` (assert `dma_pool::owns(req->buf)`), or restore real cache maintenance.

[Medium] AArch64 1 ms tick and `hrclock` still consume raw `CNTFRQ_EL0`, the value AArch32 just stopped trusting — `include/timer_arm.hpp:18-24`, `include/.../port/os-inlines.h:301-311`

```
inline std::uint32_t get_freq() { std::uint64_t freq; __asm__ ... "mrs %0, cntfrq_el0"; ... }  
...  
clock_highres::input_clock_frequency_hz() { return timer_arm::get_freq(); }  
clock_highres::cycles_per_tick()          { return timer_arm::get_freq() / 1000; }
```

and `test/boards/rpi-zero-2w/src/rtos/port_isr.cpp:65-66` re-arms with `timer_arm::get_freq() / 1000`. The AArch32 Pi timer was changed to *measure* the rate against the fixed BCM system timer precisely because "on this board it reports 19.2 MHz while the physical generic-timer counter actually increments at ~1 MHz" (`aarch32/.../timer_arm.cpp:16-59`). Both ports read the same physical counter and the same `CNTFRQ` on the same SoC. If that AArch32 premise is true, the AArch64 tick and high-res clock are wrong by the same ratio. **SUSPECTED** (unverifiable without hardware; the AArch64 port may have confirmed `CNTFRQ_EL0` is honest at EL1). Note the divergence itself is CONFIRMED. Fix: share one calibration/publish path.

[Medium] DWC2 driver hard-codes 512-byte bulk MPS for every non-EP0 endpoint — `usb/src/dwc2_driver.cpp:373,392`

```
uint32_t max_packet = (ep_num == 0) ? 64U : 512U;  
uint32_t pktcnt = (len == 0) ? 1U : ((len - 1U) / max_packet + 1U);
```

`start_tx` / `start_rx` ignore the negotiated speed and `ep->maxpacket`. Under `USB_FORCE_FS` (and any full-speed-only host) bulk MPS is 64, so `pktcnt` / `XFERSIZE` are programmed for 8× too much data per packet. The register-level twin gets this right with a compile-time `EP1_MPS` (`usb1_dev.cpp:119-123,160`). **CONFIRMED.** Fix: use the endpoint's `maxpacket`, or `is_high_speed() ? 512 : 64`.

[Medium] AArch64 `startup.S` hard-codes the DRAM window that AArch32 moved into board headers — `test/boards/rpi-zero-2w/src/startup.S:465-474`

```
ldr x1, =0x00080000
cmp x0, x1
b.lo .Lcoop_corrupt
#if defined(BOARD_RPI3B)
ldr x1, =0x3F000000
#else
ldr x1, =0x20000000
#endif
```

The AArch32 switch reads the same facts from the board header (`aarch32/src/context_switch.cpp:53-58`: `PORT_RAM_BASE`/`PORT_RAM_END`), and its own comment explains why: "They used to be the Pi's literals in this shared file, which on any other board is a check against the wrong memory map". The AArch64 port re-introduced the literals and a `BOARD_*` `#if` in the shared file. **CONFIRMED** (maintainability; a new AArch64 SoC silently validates against the wrong window). Fix: paste `PORT_RAM_BASE/END` as AArch32 does.

[Medium] DWC2 forces the (level-sensitive) USB SPI to edge-triggered and re-implements the GIC — `usb/src/dwc2_driver.cpp:338-342`

```
// Set interrupt to edge-triggered (binary pattern: 10 on 2-bit field)
*icfgr = (*icfgr & ~(3U << bit_shift)) | (2U << bit_shift);
```

The shared GIC driver treats SPIs as level-triggered (`aarch32/test/boards/luckfox-lyra/include/gic.hpp:182-186` sets every `ICFGR` field to 0), and the DWC2 controller interrupt is level/active-high. Forcing edge can drop a still-asserted interrupt. `configure_gic()` also duplicates GICD/GICC addresses and bit layouts that already exist in `gic.hpp`. **SUSPECTED** (hardware trigger type) + **CONFIRMED** duplication. Note it only runs in the non-`USB_UNDER_RTOS` standalone path (`dwc2_driver.cpp:189-196`).

[Medium] `clock_highres::cycles_since_tick()` measures against `freq/1000`, but the ISR reloads `freq/1000/OS_SYSTICK_DIV` — `include/.../port/os-inlines.h:313-326` VS `test/boards/rpi-zero-2w/src/rtos/port_isr.cpp:88`

`cycles_per_tick()` returns `timer_arm::frequency() / 1000` while the timer is re-armed with `timer_arm::period_cycles() = frequency()/1000/OS_SYSTICK_DIV`. With `-DOS_SYSTICK_DIV=2` the "cycles since tick" fraction is computed against a period twice as long as the hardware's, so the sub-tick conversion saturates or scales wrong. **CONFIRMED** (only bites the `OS_SYSTICK_DIV>1` builds).

[Medium] AArch64 has no GIC implementation at all, yet exposes `OS_SMP_IPI_SGI` —

`include/.../port/os-c-decls.h:55-56` VS `aarch32/test/boards/luckfox-lyra/include/gic.hpp`

AArch64 defines `#define OS_SMP_IPI_SGI 0` and the AArch32 GIC port consumes it, but AArch64 carries no `gic.hpp`/CPU-interface code and its BCM boards use the local mailbox for IPIs. A future AArch64 GIC board must re-derive the whole controller from the AArch32 copy. **CONFIRMED** divergence (no shared GIC source; see table).

[Low] AArch32 `_Exit` / `exit_failure` loses the exit status and uses a different reason from AArch64 — `include/semihosting.hpp:126-131`, `src/semihosting-`

`exit.cpp:18-32`

```
inline void exit_failure () { constexpr int adp_stopped_internalerror = 0x20023; exit
(adp_stopped_internalerror); }
```

versus AArch64, which passes `{0x20026, status=1}` (`aarch64/include/semihosting.hpp:82-87`). AArch32 never conveys the numeric exit code (it collapses every non-zero value to one reason), and its failure reason differs from its sibling. **CONFIRMED** divergence. Fix: `exit(0x20026, code)` in both.

[Low] AArch32 `QEMU_BUILD` still passes the `SYS_EXIT` reason by value where every other path passes a block — `include/semihosting.hpp:58-60,101-107`

The non-QEMU AArch32 path now correctly passes `volatile uint32_t block[2] = { code, subcode }` and AArch64 always does. The `QEMU_BUILD` path keeps `register int r1 ... = code;` (reason as a bare value). If QEMU follows the modern two-field pointer ABI, that path is wrong (the first pass notes the run is still judged by the printed `RESULT:` line). **SUSPECTED**; the asymmetry is **CONFIRMED**.

[Low] AArch32 `port_cpu_id()` and the assembly disagree on the Aff0 mask — `test/boards/rpi-zero-2w/src/port_sys.cpp:41`

```
return mpidr & 3u;
```

while the shared asm masks with `#0xFF` (`src/handlers.cpp:203`, `src/context_switch.cpp:66`) and the luckfox helper returns `mpidr & 0xFF`. On BCM2837 (Aff0 ∈ 0..3) it is correct today, but a board with more cores would alias. **CONFIRMED** inconsistency.

[Low] AArch32 `context::create` leaves the frame's `pad` word uninitialised — `src/rtos/os-core.cpp:350-359`

Only `vfp[]`, `vfp_hi[]` and `fpscr` are written; `ctx_t::pad` (`include/port_ctx.hpp:33`) is untouched, while AArch64 zeroes its `pad` (`os-core.cpp:329-331`). The restore path pops it into `r3`

then immediately overwrites `r3` (`handlers.cpp:228,241`), so it is harmless, but it is an uninitialised read. **CONFIRMED** (Nit).

[Low] Fault dumps dereference the faulting PC and can nest-fault — `src/exception_handler.cpp:222-224`, called at `:400`

```
volatile std::uint32_t* ptr = reinterpret_cast<volatile std::uint32_t*>(address & ~3);
std::uint32_t instruction = *ptr;
```

`handle_data_abort()` calls `print_instruction_at(fault_pc)`. If the page containing the faulting *instruction* is itself inaccessible (e.g. an abort while executing from a page that the same mapping fault removed), the read re-takes a data abort inside the abort handler. `handle_prefetch_abort()` correctly omits it. **SUSPECTED** (depends on the fault class).

[Low] Both AArch32 startups install `VBAR` without an `ISB` — `test/boards/rpi-zero-2w/src/startup.S:86-88`, `test/boards/luckfox-lyra/src/startup.S:53-55`

```
ldr r0, =exception_vectors
mcr p15, 0, r0, c12, c0, 0
```

A context-synchronizing `ISB` is required before the new vector base can be relied upon. The AArch64 `el1_entry` does `msr vbar_el1 + isb` (`startup.S:259-262`). **CONFIRMED** (window is tiny; still a spec violation).

[Low] Luckfox secondary release has no readiness handshake at all —

`test/boards/luckfox-lyra/src/smp.cpp:13-35`

Unlike the Pi boards, `start_secondary_cores()` writes the mailbox and clears the CRU resets for both cores, then returns without ever checking `_port_core_alive` (the Pi versions poll it, silently on timeout). A dead secondary is only noticed much later as a missing `g_core_stage`/heartbeat. This *sharpens* the first pass's "silent secondary-release timeout": on the luckfox there is not even a bounded wait. **CONFIRMED**.

[Low] The two AArch32 timer backends disagree about whether `CNTFRQ` may be written — `test/boards/luckfox-lyra/src/startup.S:40-42`, `.../timer_arm.cpp:24-31`

The luckfox startup writes `CNTFRQ` (`mcr p15,0,r0,c14,c0,0`) and its `init()` falls back to writing 24 MHz, while the rpi timer's comment asserts "writing it from EL1/SVC is UNDEFINED" (`rpi-zero-2w/src/timer_arm.cpp:16-19`). The shared header still exports `set_freq()` (`include/timer_arm.hpp:37-39`), so one board depends on an operation another documents as illegal. **CONFIRMED** divergence; whether the A7 ignores the write is **SUSPECTED**.

[Low] Wrong DWC2 `DCFG` address in the board's vendor gadget — `usb/src/usb_vendor_gadget.cpp:449`

```
static constexpr uintptr_t DCFG_ADDR = 0xFFB00000U + 0xC700U; // 0xFFB0C700
```

The controller base is `USB_OTG0_BASE = 0xFF740000` (`usb/include/dwc2_regs.hpp:15`), so `DCFG` is `0xFF740800`, not `0xFFB0C700`. `apply_pending_address()` is `[[maybe_unused]]` and the board's `usb_vendor_gadget.cpp` is documented as replaced by every test (`test/boards/luckfox-lyra/board.cmake`: "dma_pool.cpp, usb_env_stateos.cpp and usb_vendor_gadget.cpp are the three every test replaces"), so this is dead code. **CONFIRMED but Low**. If the file is ever built, `SET_ADDRESS` deferral writes an unrelated register.

[Low] AArch32 GIC `set_target` / `set_priority` do unlocked read-modify-write — `test/boards/luckfox-lyra/include/gic.hpp:225-249`

Byte fields are updated with `val = *reg; val &= ...; val |= ...; *reg = val;` with no lock. Two cores configuring targets concurrently lose one field. Not called concurrently today. **SUSPECTED** (latent).

[Low] AArch64 `validate_context` discards the diagnostic the AArch32 version prints — `test/boards/rpi-zero-2w/src/rtos/port_isr.cpp:33-35`

```
uart::uart1 << "\n!!! CORRUPT STACK POINTER !!!\n";  
(void)old_sp;
```

AArch32 prints `new_sp`, `old_sp` and the decoded CPSR (`aarch32 port_isr :34-53`). On the 64-bit port a tripwire gives strictly less evidence. **CONFIRMED** (Nit). Also, like the fatal path, these prints are semihost-mirrored (see finding 1).

[Low] AArch64 semihosting clobber lists are weaker than AArch32's — `include/semihosting.hpp:38,47,60,72` VS `aarch32/include/semihosting.hpp:77,90,107,115`

AArch32 declares `"r2", "r3", "ip", "lr", "memory", "cc"`; AArch64 declares only `"memory", "cc"`. The AArch64 `hlt` ABI only promises `x0` is a result, but if a handler clobbers `x2` / `x3` the compiler may hold live values there. **SUSPECTED** (Nit).

[Low] AArch64 has a hand-written `memset` to dodge `DC ZVA`; AArch32 does not — `test/boards/rpi-zero-2w/src/port_sys.cpp:28-82`

AArch64 replaces newlib's `memset` because "newlib's optimized AArch64 memset clears big buffers with DC ZVA, which on the real Cortex-A53 FAULTS when executed before the MMU and caches are enabled", and it also had to defeat `-ftree-loop-distribute-patterns` recursion. AArch32 has no equivalent guard. The AArch64 explanation (SCTLR_EL1.DZE / pre-MMU `DC ZVA`) is sound and its `SCTLR_DZE` is

set in `mmu.cpp:62,155`; the AArch32 port runs the same `memset` over `.bss` before `mmu_init` with caches off. **SUSPECTED** (no reported AArch32 failure, but the asymmetry is worth a note).

[Nit] AArch64 silently falls back to 19.2 MHz; AArch32 forwards a raw 0 to callers

— `include/timer_arm.hpp:16-24` (A64) vs `include/timer_arm.hpp:41-45` (A32)

`get_freq()` differs in contract (A64 masks `0` to a fallback, A32 returns raw `0`; the A32 board `init()` handles the `0`). Minor API divergence.

AArch32 vs AArch64 divergences

Beyond the unavoidable ISA splits (banked modes/VFP vs PSTATE/Q-regs, CP15 vs system registers, short-descriptor vs 4-level MMU), these are duplicated or diverged facts that should have been shared:

Topic	AArch32	AArch64	Where
Context-switch asm	<code>src/context_switch.cpp</code> + <code>handlers.cpp</code> (<code>ctx_t</code> , 82 words)	<code>startup.S</code> (<code>ctx64</code> , 800 B)	frame layouts are ISA- specific; bounds check , tripwire , publish protocol duplicated
Frame bounds in switch	board macros <code>PORT_RAM_BASE/END</code>	hard-coded <code>0x80000</code> / <code>0x20000000</code> / <code>0x3F000000</code>	A32 <code>context_switch.cpp:53-58</code> ; A64 <code>startup.S:465-474</code>
Exception entry	banked UND/ABT/SVC frames, per-mode stacks	single EL1h <code>SAVE_FRAME</code> , <code>SP_EL1</code>	A32 <code>handlers.cpp:50-145</code> ; A64 <code>startup.S:27-68</code>
Exception reporting	rich abort/undef dump + test- mode resume, UART-only	minimal ESR/ELR/FAR dump, semihost-mirrored	A32 <code>exception_handler.cpp</code> ; A64 <code>exception_handler.cpp:42-65</code>
MMU build/enable	short descriptor, <code>mmu_init</code> / <code>mmu_enable</code>	4-level, same split	both <code>boards/.../src/ mmu.cpp</code>
Timer rate	Pe: measured/calibrated; Lyra: trusted + write fallback	raw <code>CNTFRQ_EL0</code> , fallback const	A32 <code>timer_arm.cpp</code> (per board); A64 <code>timer_arm.hpp:18-24</code>
Timer API	<code>frequency()</code> , <code>period_cycles()</code> , <code>OS_SYSTICK_DIV</code>	none of these	A32 <code>include/ timer_arm.hpp:77-86</code>
Tick phasing	<code>OS_SYSTICK_DIV</code> phase counter	ignored	A32 <code>port_isr :95-101</code> ; A64 <code>port_isr :65-73</code>
GIC	GIC-400 driver (<code>gic.hpp</code> , luckfox)	none, mailbox IPI only	A32 only
<code>OS_NCPU</code> /SGL contract	<code>#error</code> unless from board.cmake	silent <code>4</code> / hard-coded <code>0</code>	A32 <code>os-c- decls.h:58-63</code> ; A64 <code>:52-56</code>
IRQ state type	<code>uint32_t</code>	<code>uint64_t</code> (DAIF)	both <code>os-c-decls.h</code>
Semihosting trap	<code>SVC 0x123456</code> / HLT chosen per board	always <code>HLT #0xF000</code>	both <code>include/ semihosting.hpp</code>
<code>SYS_EXIT</code> param	HW/OpenOCD: 2-word block; QEMU: by value	always 2-word block	A32 <code>:101-116</code> ; A64 <code>:66-73</code>
<code>SYS_EXIT</code> failure	reason <code>0x20023</code> , no status	<code>{0x20026, 1}</code>	A32 <code>:126-131</code> ; A64 <code>:82-87</code>
USB/DWC2	full gadget stack (luckfox)	absent	A32 <code>test/boards/ luckfox-lyra/usb/</code>
<code>switch_stacks</code> lock	inline duplicate of <code>_smp_klock_raw_acquire</code>	inline duplicate of LDAXR helper	both <code>rtos/os-core.cpp</code>

Topic	AArch32	AArch64	Where
<code>_Exit</code> location	<code>src/semihosting-exit.cpp</code>	<code>src/handlers.cpp</code>	both

Agreement with first pass

- **AGREE, now FIXED in the tree** — `clock_highres` raw `CNTFRQ`. The first pass's top finding was real. In the current tree `aarch32/include/.../os-inlines.h:307-311` returns `timer_arm::frequency()` (calibrated), and commit `cfec772` says so. The **new** point is that AArch64 was never given the same fix (finding above) — the premise ("CNTFRQ lies on this SoC") applies to the same silicon.
- **AGREE, now FIXED** — AArch32 `SYS_EXIT` bare reason. `include/semihosting.hpp:109` now passes `volatile uint32_t block[2]`. The residual asymmetry is the `QEMU_BUILD` by-value path and the failure-reason divergence (findings above).
- **AGREE, now FIXED** — AArch64 `T1SZ=0` /TTBR1 walks. `mmu.cpp:139-147` now sets `(1ULL << 23)` (`EPD1`) with an explicit comment; commit `04a6b4b`.
- **AGREE** — `OS_HAS_INTERRUPTS_STACK` is not honoured. I add confirming evidence from the *consumer* side: tests honour the macro by installing the kernel's interrupt stack (`aarch32/test/luckfox-lyra/smp_test0/main.cpp:52-56` calls `os::rtos::interrupts::stack()->set(__fiq_stack_top, ...)`), so the port advertises a capability the ISR path does not use.
- **AGREE** — `-00` **silent secondary-release timeout**. Plus a sharper variant: the luckfox port has no bounded wait at all (finding above).
- **AGREE** — `-00` **only DWC2**. I can now name a concrete mechanism (unenforced NC-buffer contract + no-op cache maintenance) rather than only the symptom; the exact `-02` regression line remains unproven.
- **AGREE** — AArch64 **board-fact defaults**. Confirmed verbatim at `aarch64/include/.../os-c-decls.h:52-56`.
- **AGREE** — AArch64 ignores `OS_SYSTICK_DIV`. Still true; I add that `cycles_since_tick()` is wrong under the same builds.
- **AGREE** — **Pi timer calibration runs per core with IRQs masked**. Still true (`timer_arm.cpp:50-54` called from every core's `port_sys_init`).
- **AGREE** — AArch32 TTBR0 IRGN is WB, not WBWA. Still true (`rpi mmu.cpp:96-101`).
- **PARTIAL/DISAGREE** — AArch64 spin-table `0xD8 + 8*core`. The first pass already marked this unverifiable; I likewise could not confirm it. It stays a hardware check, not a finding.
- **DISAGREE (baseline drift)** — **the three "fix" findings above are no longer present**. A re-run of the first pass against this tree would move them from "High/Medium open" to "confirmed and resolved"; the remaining exposure is the AArch64 side, not AArch32.

Good practices

- **Fault console isolation** on AArch32 (`exception_handler.cpp:15-113`): `puts_uart` / `putc_uart` only, with a precise rationale. This is the right pattern and should be ported to AArch64 (finding 1).
- **Deferred publish + claim-by-null + tripwire**: the `_smp_pub_*` staging, the `stack_ptr = nullptr` claim (`os-core.cpp:288-297` / `aarch64:269-273`), and the bounds/SPSR-CPSR tripwires turn a protocol violation into an on-the-spot halt with evidence.
- **Kernel-lock release ordering** is documented and correctly ordered (clear `owner` / `depth` before the lock word), with the prior total-freeze bug recorded (`os-core.cpp:302-322` / `aarch64:278-299`).
- **Cache maintenance before a caches-off receiver**: `DCCMVAC` / `dc cvac` + `dsb` before `sev` / mailbox, with the coherency reasoning in comments (`rpi smp.cpp:49-60` , `aarch64 smp.cpp:53-59`).
- **Semihosting trap treated as a property of the debugger**, selected per board, with the HLT-on-ARMv7 hazard explicitly called out and avoided for the A7 (`semihosting.hpp:10-34`).
- **AArch64 `memset` override** correctly prevents pre-MMU `DC ZVA` faults and GCC pattern-distribution recursion, in pure asm (`aarch64/.../port_sys.cpp`).
- **Board-fact centralisation** in `board.cmake` (`OS_NCPU` , `OS_SMP_IPI_SGI` , `PORT_RAM_*` , `PORT_GREETING`) with `#error` enforcement — on AArch32. The AArch64 port should adopt the same contract.

Second-pass review: µOS++ Ille Cortex-M & POSIX ports

Scope

Read-only, second independent pass over:

- `/home/dan/Work/micro-os-plus-iii-cortexm.git/{src,include,include-m33,include-rp2350,test/boards/*}`
- `/home/dan/Work/micro-os-plus-iii-posix-arch.git/{src,include,test}`

Reviewed tree state: Cortex-M HEAD `18d91f1` ("fix(port): release klock on null thread in switch_stacks, mask IRQs during context switch, and restore RP2350 PRIMASK"), POSIX HEAD `0aab776`. The Cortex-M tree is clean, so the first pass was performed on the parent of `18d91f1` and several of its findings are already addressed there. The shared kernel (`micro-os-plus-iii-smp.git`) was read for reference (`internal_switch_threads` , `port/smp-common` , `semihosting.h`), but the findings below are in the two named trees unless stated.

Not built (no tree-changing builds per instructions); RP2350 bootrom/bootrom details and the macOS leg are therefore marked unverifiable/suspected.

Summary

The scheduler logic is functionally sound on the paths the hardware tests exercise, and the recent `18d91f1` closes the four worst first-pass defects (lock-while-spinning, IRQ unmasked across the picker, RP2350 `port_put_lock(0)`, dead high-res overflow). The dominant *new* conclusions are structural and borderline: (1) three near-identical scheduler implementations (`os-core.cpp`, `os-core-m33.cpp`, `os-core-rp2350.cpp`) now drift in ways that a single shared source would prevent; (2) the POSIX port's macOS leg is not merely process-directed but almost certainly unbuildable (`timer_create` absent on Darwin) and, even with a shim, can starve the kernel clock; (3) the test verdict helper's hand-rolled semihosting contradicts the kernel's own clobber contract. The rest are Medium/Low robustness and UB items, plus two genuine test-runner robustness gaps where the newer `run-hw.sh` already does the right thing and the per-board `hw.sh` copies do not.

New/updated findings

[High] Three scheduler cores triplicate ~88% and have begun to drift —

`src/rtos/os-core{, -m33, -rp2350}.cpp`

Comment-stripped, non-blank line counts: `os-core.cpp` 416, `os-core-m33.cpp` 566, `os-core-rp2350.cpp` 495. m33↔rp2350: 438 byte-identical lines and `diff` reports 205 differing lines — i.e. ~77% of m33 / ~88% of rp2350 is the same text; every one of `context::create`, `clock_systick::start`, `save_on_stack`, `restore_from_stack`, `switch_stacks`, `reschedule`, `locked`, `start`, and the `frame_t` layout is copied three times. Drift already present:

- `os-core.cpp:309-310` `f->r15_pc = ... & (~1)` vs `os-core-m33.cpp:203-204` and `os-core-rp2350.cpp:150-151` `... | 1`.
- `os-core.cpp:313-321` honours `OS_B00L_RTOS_PORT_CONTEXT_CREATE_ZERO_LR`; m33/rp2350 dropped the option and always write `func + 2`.
- m33/rp2350 enable FPU stacking at `start()`; `os-core.cpp:439-442` only comments about it.
- `_smp_klock` is a 3-field `{lock, owner, depth}` in `include-m33/.../os-decls.h:121-126` but 2-field `{owner, depth}` in `include/.../os-decls.h:133-137` and `include-rp2350/.../os-c-decls.h:52-56`; the initializers (`os-core-m33.cpp:247` `{0, SMP_NO_OWNER, 0}` vs `os-core-rp2350.cpp:195` `{SMP_NO_OWNER, 0}`) are shape-matched by hand and only a compile error protects them.

Which is authoritative: `os-core.cpp` is the upstream reference for the single-core frame layout and `context::create` (and the only one handling `__ARM_ARCH_6M__` and `OS_INTEGER_RTOS_CRITICAL_SECTION_INTERRUPT_PRIORITY`); `os-core-rp2350.cpp` is the de-facto authoritative SMP core (it is the only one with hardware-validated tests); `os-core-m33.cpp` is a QEMU-only clone of it. Recommend a shared `.inc` /template parameterised on cpu-id/IPI/lock/launch, as the header comments themselves claim ("the RP2350 port's analogue").

[High] POSIX macOS leg cannot build and can starve the kernel clock — `src/`

`host_cpu.cpp:212-231`, `src/diag/trace-posix.cpp`, `include/cmsis-plus/rtos/port/os-decls.h:174-184`

`arm_tick()` uses `timer_create(CLOCK_MONOTONIC, ...)` / `timer_settime()` unconditionally; the only platform split is the notification mode:

```
#if defined(__linux__)
    sev.sigev_notify = SIGEV_THREAD_ID;
    ...
#else
    sev.sigev_notify = SIGEV_SIGNAL;
#endif
```

Darwin/libSystem does not provide POSIX per-process timers, so the `__APPLE__` branch is unverified and would `fatal("timer_create")` (or fail to link). Even granting a shim, `SIGEV_SIGNAL` is *process-directed*: the timer can be delivered to any host thread that does not block `SIGRTMIN`, and `tick_handler` (`host_cpu.cpp:152,164`) then evaluates `cpu == 0` on the *receiving* thread. If the tick lands on a host thread whose `_this_cpu != 0`, `os_systick_handler()` is skipped and the kernel clock does not advance. This is sharper than the first-pass note (misdirected tick): it is a potential total clock stall, and the macOS target appears unbuildable anyway. I AGREE with the first pass and escalate.

[Medium] `hw_result::semi_write0` lies about its clobbers — `test/boards/shared/`

`hw_result.hpp:38-44`

```
register unsigned r0 __asm__ ("r0") = 0x04u; // SYS_WRITE0
register const char* r1 __asm__ ("r1") = s;
__asm__ volatile ("bkpt 0xAB" : : "r" (r0), "r" (r1) : "memory");
```

The kernel's own trap (`micro-os-plus-iii-smp.git/include/cmsis-plus/arm/semihosting.h:117`) declares `"r0", "r1", "r2", "r3", "ip", "lr", "memory", "cc"` clobbered for the same `bkpt 0xAB` call, following libgloss. The test helper declares only `memory`. If the semihosting service corrupts any of `r2/r3/ip/lr` (the conservative assumption the rest of the project makes), this is a latent miscompile on the exact path (`RESULT: PASS`) every hardware runner greps. The two copies should be one, or the helper should list the same clobbers.

[Medium] `hw_result` always traps before exit, so "no semihosting" is a fault, not an idle loop — `test/boards/shared/hw_result.hpp:19-20, 50-59`

The doc says "The idle loop that follows the call is therefore only reached when semihosting is NOT available", but `ok()` / `fail()` call `semi_write0()` (a raw `bkpt 0xAB`) *before* `std::exit`, with no `#if defined(SEMIHOST)` guard. A bare image without a semihosting-enabled debugger HardFaults at the `bkpt` and never reaches the documented idle fallback. Here every board sets `SEMIHOST` (`test/`

`CMakeLists.txt:68 set(_common_defines TRACE SEMIHOST)`) and every `hw.sh` enables semihosting, so the claim is currently unreachable — but the comment is wrong and the guard the sibling ports rely on is absent.

[Medium] POSIX `clock_highres` timestamp is a shared, non-atomic, 32-bit-wrap counter — `src/rtos/os-core.cpp:561-604`

`static uint64_t previous_timestamp;` is file-global, read/written by `cycles_per_tick()` and `cycles_since_tick()` with no lock or atomic. On an SMP test the 64-bit store can be observed torn on a 32-bit host, and the `uint32_t` deltas wrap after ~71 minutes of monotonic time (`clock_highres::input_clock_frequency_hz()` is 1,000,000, so the unit is μ s). The RP2350/m33 ports avoid this by using a real hardware counter; the POSIX fallback should either take the `_smp_tlock` / `port_tmr_lock` or use `now()`'s own `clock_gettime`.

[Medium] Per-board `hw.sh` reports every OpenOCD failure as a 600 s TIMEOUT — `test/boards/pico2/hw.sh:99-118` (same in `pico2-pizero`, `pico2-rp2350b-psram`)

The wait loop only checks `RESULT: PASS/FAIL` and `kill -0`; an OpenOCD that dies immediately (`kill -0` false) falls through with `rc=2` and prints "TIMEOUT after 600s", and there is no `^Error: / "Invalid ACK"` detection. The shared newer runner `micro-os-plus-iii-smp.git/test_smpl/run-hw.sh:234-260` already distinguishes `OPENOCD DIED` (4), `OPENOCD ERROR` (5) and `DEBUG LINK LOST` (6) and tells the operator to lower the adapter clock. The board copies should adopt the same classification; a probe failure otherwise costs ten minutes per mis-diagnosed run.

[Medium] RP2350 `switch_stacks` fatal path silently parks, and diverges from POSIX — `src/rtos/os-core-rp2350.cpp:454-468`, `src/rtos/os-core-m33.cpp:476-488`, `src/rtos/os-core.cpp:906-907`

The post-`18d91f1` null-`new_thread` path now releases `_smp_klock` (good), but then masks IRQs with `cpsid if` and `wfi` s forever on that core. That is a real kernel invariant violation (the picker is documented to fall back to `os_idle_thread_core[cpu]`, `os-core.cpp:621-629`) and it is now silent on both ARM SMP cores, whereas the POSIX port treats the same condition as fatal and `::abort()` s (`src/rtos/os-core.cpp:332-338`). The upstream `.cpp` has no guard at all and would null-deref, and it was *not* given the same fix, so the three cores now disagree on what a null pick means. At minimum log it via `g_boot_mark` /the fault beacon; ideally the invariant should be asserted.

[Medium] Inline-assembly save/restore clobbers callee-saved registers but declares no clobber — `src/rtos/os-core-rp2350.cpp:560-609`, `os-core-m33.cpp:572-609`, `os-core.cpp:677-813`

`save_on_stack` / `restore_from_stack` write r4-r11 and lr but list only the SP output/input and explicitly say "DO NOT add anything here!". They are safe only because they are inlined into a `naked` handler that uses those registers for nothing else. This is a carried-forward upstream fragility, not new, but it is now triplicated, and the FPU branch is new: `vstmdbeq / vldmiaeq %[r]!, {s16-s31}` also depends on the compiler having chosen `%[r]` freely, so a future GCC that allocates `%[r]` to r4 (a member of the

following `stmdb` list) would corrupt the frame. Worth a compile-time assertion/`asm("r0")` pin rather than a comment in three files.

[Medium] `m33` `save_on_stack` does not mask IRQs before saving, and its lock acquisition order differs from the documented rule — `src/rtos/os-core-`

`m33.cpp:452-521`

The new masking is inside `switch_stacks`, i.e. after `save_on_stack()` has run in `PendSV_Handler`. That is fine for the picker (the shared list), but the comment claims IRQs are masked "around context switch", which is not literally true for the `save_on_stack` window. More importantly, `m33/rp2350` acquire the recursive lock by a hand-inlined

```
if (_smp_klock.owner != cpu) { _smp_klock_raw_acquire (); _smp_klock.owner = cpu; }
_smp_klock.depth = _smp_klock.depth + 1;
```

while `port_set_lock()` (`include-rp2350/.../os-inlines.h:166-176`, `include-m33/.../os-inlines.h:170-183`) does the same but *returns* the saved PRIMASK. Having two implementations of the same recursive lock, one of which does not publish `lock_primask`, is a maintenance hazard that the "one lock discipline" comments cannot enforce.

[Medium] `_getentropy` is deterministic, has no length bound and mishandles zero-length — `src/libc/getentropy.c:19-35`

```
p[i] = (uint8_t)(0xA5u ^ (uint8_t)i);
```

Every call returns the same stream. It is honest about being a placeholder for emulated targets, but newlib's contract is `getentropy(buf, len)` with `len <= 256` and `EIO`/`EINVAL` otherwise; there is no bound check, and `buffer == NULL` is rejected even for `len == 0` (where POSIX permits it). Low risk on bare-metal, but it is the kind of "random" a test could accidentally trust.

[Low] `context::create` alignment uses `reinterpret_cast<int>` on pointers — `src/rtos/os-core.cpp:289-293,357`, `os-core-m33.cpp:189-192`, `os-core-rp2350.cpp:136-139`

```
p = reinterpret_cast<...>((reinterpret_cast<int> (p)) & (~3));
...
assert (((reinterpret_cast<int> (&f->r0)) & 7) == 0);
```

Casting a pointer to `int` is 32-bit-only. It is guarded by `__ARM_EABI__` today, but it would silently truncate if these `#include` /port files were ever built for an LP64 host (the POSIX port reuses the same naming and nearly the same code). `uintptr_t` is the correct type and the file already includes `<cstdint>` via `os.h`.

[Low] POSIX `critical_section::exit` can unmask an IPI that was deliberately blocked — `include/cmsis-plus/rtos/port/os-inlines.h:197,216`

The saved state is a single bool recording only `clock::signal_number()` (the tick). `exit()` then does `pthread_sigmask (state ? SIG_BLOCK : SIG_UNBLOCK, &irq_set, ...)`, which manipulates *both* the tick and the IPI signal. If only the IPI was blocked on entry, the exit unblocks it too. The save state should be the full old `sigset_t` intersection with `irq_set`, as `in_handler_mode()`'s neighbours already imply.

[Low] RP2350 `port_smp_secondary_start` uses raw FPCCR address while m33 uses the CMSIS symbol — `src/rtos/os-core-rp2350.cpp:687-688` VS `os-core-m33.cpp:64, 275`

```
rp2350::reg (0xE000EF34) |= 0xC0000000u; // FPCCR: set ASPEN|LSPEN
```

vs m33's `FPU->FPCCR |= (FPU_FPCCR_ASPEN_Msk | FPU_FPCCR_LSPEN_Msk)`. The magic address duplicates CMSIS and will not be updated if the core changes; the `0xC0000000` mask likewise duplicates the bit definitions. `rp2350::reg` also reads-modify-writes a register that on a core without an FPU does not exist, though the file is M33-only.

[Low] m33 sets VTOR in the secondary entry; rp2350 relies on the bootrom — `src/rtos/os-core-m33.cpp:681`, `os-core-rp2350.cpp:675-693`

m33's `port_smp_secondary_start` does `SCB->VTOR = __interrupt_vectors`; rp2350's does not, because `multicore::launch_core1` passes `__vector_table` as the bootrom's launch VTOR. Both are correct for their machine, but the asymmetry is undocumented in the rp2350 function and makes the two files harder to compare (or unify). If the RP2350 launch path is ever changed to a non-bootrom release, VTOR will be wrong.

[Low] `launch_core1` readiness guard can be outrun and silently proceeds — `test/boards/pico2/glue/multicore.cpp:108-138`

The bootrom readiness wait breaks out after 2,000,000 iterations (`guard > 2000000U`) and then enters the echo handshake regardless. The comment argues the sequence loop handles the "already booted" case, but a genuinely slow core-1 boot that exceeds the guard would start pushing commands into a FIFO the bootrom may still be draining, and the handshake can livelock. The guard is unbounded on the wrong side: it converts a hang into a *possible* silent bad state rather than retrying the reset. Timing not verifiable here.

[Low] `pkill -9 -f "openocd.*boards/pico2/openocd.cfg"` is over-broad — `test/boards/pico2/hw.sh:73` (+ pizero/rp2350b copies)

Matches any process whose command line contains that pattern, not necessarily the intended OpenOCD; and it is run unconditionally before each test, so a concurrently running different test/board could be killed. `run-hw.sh:126` instead only *warns* when an OpenOCD is already running. Prefer recording the PID or checking the exact binary + config.

[Low] Stale file path in runner/doc comments — `test/boards/pico2/hw.sh:15` and siblings

Comments point at `test/pico2/include/hw_result.hpp`; the file actually lives at `test/boards/shared/hw_result.hpp` (the only copy; `test/CMakeLists.txt:173-177` adds `boards/shared` to every board's include path). Same stale path in the pizero and rp2350b scripts and in `hw_result.hpp`'s own prose. Nit, but these paths are what an operator greps at 2 a.m.

[Low] `reschedule()` is called with interrupts enabled and relies on the caller's mask

— `src/rtos/os-core.cpp:615-650`, `os-core-m33.cpp:432-445`, `os-core-rp2350.cpp:387-404`

All three optimise away the `is_reschedule_pending`/instrumentation bookkeeping differently: upstream sets `scheduler::is_reschedule_pending = true` and calls `instrumentation::scheduler::reschedule()`; m33 and rp2350 do neither. If any instrumentation or validator depends on `is_reschedule_pending`, it silently does nothing on the M33/RP2350 builds. No caller in the reviewed trees reads it, so this is currently latent, but it is more triplication drift.

[Nit] RP2350 `switch_stacks` has a stray double blank line and ad-hoc globals — `os-`

`core-rp2350.cpp:396-397, 486-509`

Trivial, except that `g_sw_*`/`g_boot_mark` are `extern "C"` globals the port writes on every context switch; m33 defines local no-op stubs while rp2350 requires the BSP to define them. A missing BSP symbol becomes a link error, which is intended, but the divergence (local definition vs hard dependency) is another reason the two files should not be separate copies.

Agreement with first pass

- SMP `switch_stacks` "no ready thread" spinning while holding the kernel lock — **AGREE** it was real; **CONFIRMED FIXED** by `18d91f1` (`os-core-m33.cpp:476-488`, `os-core-rp2350.cpp:454-468` now release `_smp_klock` before `wfi`). Residual: the core still parks silently instead of asserting (finding above), and `os-core.cpp` never got the guard.
- RP2350 core losing M33's PRIMASK save/restore and calling `port_put_lock(0)` — **AGREE**; **CONFIRMED FIXED** by `18d91f1` (`lock_primask[OS_NCPU]` added at `os-core-rp2350.cpp:194`, published at `:344`, used at `:354`; declared in `include/.../os-decls.h:126`). The M33 already returned/used PRIMASK (`os-core-m33.cpp:381-408`, `port_set_lock` at `include-m33/.../os-inlines.h:170`).
- Neither core masking local IRQs around the thread picker — **AGREE**; **CONFIRMED FIXED** by `18d91f1` (`__get_PRIMASK`/`__disable_irq` at `os-core-m33.cpp:455-456`, `os-core-rp2350.cpp:427-428`, restored at `:519`/`:504`). Caveat: the mask starts inside `switch_stacks`, not around all of `PendSV_Handler`; that is sufficient for the shared ready list but the comment overstates it.
- Dead high-res overflow test — **AGREE**; **CONFIRMED FIXED**. All three headers previously tested `SysTick->CTRL & SCB_ICSR_PENDSTSET_Msk` (bit 26 does not exist in `SysTick->CTRL`), so the overflow branch was unreachable. `18d91f1` changed them to `(SCB->ICSR &`

`SCB_ICSR_PENDSTSET_Msk) != 0` (`include/.../os-inlines.h:463`, `include-m33/...:397`, `include-rp2350/...:511`). The corrected arithmetic itself is unchanged and still non-atomic across the two `SysTick->VAL` reads (Low).

- **POSIX macOS tick being process-directed — AGREE, and I escalate:** the `SIGEV_SIGNAL` path (`host_cpu.cpp:223`) is process-directed *and* keys `os_systick_handler()` off the receiving thread's `_this_cpu`, so it can stop the clock, not merely misattribute a tick. Additionally `timer_create` is not available on Darwin, so the `__APPLE__` leg is likely unbuildable (unverifiable here — no macOS toolchain run).

Where I **DISAGREE**: I do not think the RP2350 `port_put_lock(0)` or the missing IRQ mask were the most consequential of the first pass's four — the lock-while-spinning and the dead overflow check are now demonstrably closed, whereas the three-way source duplication and the POSIX clock-stall are the defects most likely to bite next, and neither was in the first pass.

Good practices

- The post-`18d91f1` unlock ordering in `locked()` writes `lock_state/owner` *before* releasing the lock word, with the exact hazard written down (`os-core-m33.cpp:349-354`, `os-core-rp2350.cpp:350-355`, `src/rtos/os-core.cpp:360-373`); the POSIX port documents the same rule at length (`src/rtos/os-core.cpp:360-373`), and the fix is consistent across all three.
 - The RP2350 bootrom handshake is documented with the precise failure it avoids (core 1's drain discarding an early command) and it drains the FIFO before waiting (`multicore.cpp:96-137`); the SIO FIFO IRQ is deliberately left off until after the launch (`os-core-rp2350.cpp:281-295`, `:658-668`), which is the subtle ordering the pico-sdk uses.
 - `os-c-decls.h/os-decls.h` shadowing uses the *same* include guard across targets, and `_getentropy/getentropy.c` and the weak `os_board_console_mirror` hook (`src/semihosting/c-syscalls-semihosting.cpp:453-467`) are small, single-purpose seams rather than special cases in the kernel.
 - The POSIX port's deliberate rules — no native TLS across a switch point, `noinline` `errno` restore (`host_cpu.cpp:127-131`), `pthread_sigmask` not `sigprocmask`, and the full "a host thread IS a CPU" model — are unusually well argued and internally consistent; `test-smp-boot.hpp:36-43` even fixes a latent `port_cpu_id() & 3` bug for non-power-of-two core counts.
 - Board bring-up polls are generally bounded (`clocks::usb_init` returns false instead of hanging, `rtos-glue.cpp` fault handlers dump just enough state, and the pico2 `hw.sh` re-reads the log after OpenOCD exits to defeat block buffering) — the robustness gaps above are exceptions, not the rule.
-

Second-pass review — μOS++ IIIe test harness (`micro-os-plus-iii-smp.git/tests/`)

Scope

Read-only second pass over the test harness, written from the tree at `/home/dan/Work/micro-os-plus-iii-smp.git` (HEAD `a54e782`):

- `tests/CMakeLists.txt`, `tests/cmake/{tests-main,common-options,global-definitions}.cmake`
- `tests/platforms/*/CMakeLists.txt` and `tests/platforms/*/cmake/*.cmake` (22 platforms)
- `tests/device-qemu-cortexm/CMakeLists.txt`
- `tests/sources/*` (portable suites) and their `src/`
- `tests/package.json`
- `tests/README.md`, `tests/TO-CHECK.md`, `docs/tests/*`
- `tests/build/*` generated `CTestTestfile.cmake` / `compile_commands.json` (used as evidence of what the CMake actually does, not just what it says)
- `test_smpl/run-{qemu,hw,host}.sh` and the five `platforms/*/src/platform-support.cpp`
- `tools/verify-no-duplicate-sources.py` (it governs `tests/`)

No file was modified. The only file created is this report.

Summary

The second pass confirms the first pass's headline findings with harder evidence (generated CTest/compile-command output) and adds several new ones. The most important new results:

1. **The runners can report PASS for a test that never ran.** Both `run-qemu.sh` and `run-host.sh` exit 0 when `UOS_QEMU_ONLY` / `UOS_RUN_ONLY` selects an image that does not exist but other images do. Every platform test invokes them with that variable. Because the platform loops register a CTest case per source directory **without** checking the target exists (only `cortexm-pico2` checks `TARGET` for the qemu case), a renamed/removed image turns into a green test. This is the single highest-value finding.
2. **The duplicate-source gate exempts the whole `tests/` tree** with the reason "upstream's own test suite" — a reason that stopped being true when the SMP harness moved in. The five `platform-support.cpp` copies and the four near-identical `Pi CMakeLists.txt` are invisible to the one tool built to catch exactly them.
3. **`platform-support.cpp` is *not* dead — but the `micro-os-plus::platform` and `micro-os-plus::platform-support` interface targets are.** The port's builder compiles the file directly (proved

by `compile_commands.json`); nothing links the interface targets. That distinction matters: the first-pass "unused target" is real, and the live-vs-dead copy inversion (below) is the sharper consequence.

4. **The live Lyra `platform-support.cpp` is the stale copy.** The four Pi copies carry the fixes (`test_wait_secondaries(3000)`, `os_board_console_mirror`, `initialise_monitor_handles()`); the one copy the Lyra actually links lacks all three.
5. **Every hardware-only platform's `xpm run test` runs zero tests and reports success** (`ctest -V -LE hwd` over a case set that is entirely `hwd`), with no guard and no documented warning.

Overall the harness is well engineered — CTest names are unique (no double registration), the runner verdict protocol is coherent, and the `ENVIRONMENT` / `LABELS` plumbing is mostly consistent — but the "reuse the port's builder" refactor left a layer of unused targets, stale copies, and a few silent-green paths.

New/updated findings

[High] A missing image is reported as PASS — `test_smpl/run-qemu.sh:86-92,145-146` ;
`test_smpl/run-host.sh:55-62,96-97`

CONFIRMED by reading, and it is reachable. Both runners iterate the built images and then require only `[[ $fail -eq 0 ]]`:

```
# run-qemu.sh
ONLY="${UOS_QEMU_ONLY:-}"
for img in "${BUILD_DIR}"/*-qemu.bin; do
    [[ -e "$img" ]] || { echo "no *-qemu.bin in ${BUILD_DIR}"; exit 2; }
    app="$(basename "$img" -qemu.bin)"
    if [[ -n "$ONLY" && "$app" != "$ONLY" ]]; then continue; fi
    ...
done
echo "qemu suite: ${pass} passed, ${skip} skipped, ${fail} failed"
[[ $fail -eq 0 ]]
```

If the build directory holds *some* `*-qemu.bin` but not the one named by `UOS_QEMU_ONLY`, the loop skips every iteration, `fail` stays 0, and the script exits 0 → CTest marks the case **PASS**. The `[[ -e "$img" ]]` guard only fires when the glob matched nothing at all (no `nullglob`). `run-host.sh:57-62` has the identical shape with `UOS_RUN_ONLY`. Every `-qemu` / `-host` / `-hwd` case the platform CMakeLists register sets one of these variables, so the guard is always active. On a `hwd` case the equivalent hole is smaller (`run-hw.sh:288-289` requires the exact `$WHICH-hwd` file), but `ctest -LE hwd` means `hwd` is the non-CI path anyway.

Reachability: the platform loops below register a case **without** checking that the target exists, so a suite that the port's builder no longer emits (a rename, an `HWD_ONLY` / `NO_KERNEL` reclassification, a board-specific exclusion) produces a permanent green test rather than an error.

[High] The duplicate-source gate exempts `tests/` wholesale, with a stale reason

— `tools/verify-no-duplicate-sources.py:99-101`

CONFIRMED:

```
EXEMPT = [  
    (None, "tests/",  
     "upstream's own test suite, carried as shipped (spec Section 10)"),
```

The tool's whole purpose (per its own docstring) is to catch cross-repo duplication of `.c/.cpp/.h`. The SMP harness is no longer "upstream's own test suite"; `tests/` now holds newly written code, including five `platforms/*/src/platform-support.cpp` and four Pi `CMakeLists.txt` that are comment-only variations of each other. Because the exemption is a path-prefix match on the whole tree, none of them are ever reported. The exemption's reason is a claim about content that is now false — a review finding in its own right.

[Medium] `micro-os-plus::platform` / `micro-os-plus::platform-support` are unused on the 10 "reuse the port's builder" platforms — aarch32-, aarch64-, cortexm-pico2*, cortexm-nucleof411/weactf411/412

CONFIRMED with generated evidence, and it **refines** the first pass. In `compile_commands.json` for `aarch64-rpi-zero-2w` exactly one file under `tests/platforms/` is compiled:

```
/home/dan/Work/micro-os-plus-iii-smp.git/tests/platforms/aarch64-rpi-zero-2w/src/platform-  
support.cpp
```

and its object lives under the **port's** build (`platform-bin/port-tests/test/CMakeFiles/mutex-stress-hwd.dir/.../platforms/aarch64-rpi-zero-2w/src/platform-support.cpp.obj`), not under `platform-bin/CMakeFiles/platform-...`. The port's builder names the harness file directly. Nothing links `platform-aarch64-rpi-zero-2w-interface` (alias `micro-os-plus::platform`) or `platform-aarch64-rpi-zero-2w-support-interface` (alias `micro-os-plus::platform-support`); the platform CMakeLists never reference them. The same is true for `cortexm-pico2`, `cortexm-pico2-pizero`, `cortexm-pico2-rp2350b-psram`, `cortexm-nucleof411`, `cortexm-weactf411/412`, and the aarch32 family. So the first pass is right that the targets are unused — but the sharper point is:

- The Pi base target (`platforms/aarch64-rpi-zero-2w/cmake/platform-library.cmake:53` `QEMU_BUILD`, `:69` `-T${UOS_BOARD_LINKER_QEMU}`) is a **landmine**, not merely dead: it is the Qt emulator variant with the QEMU linker script, and the moment somebody links it for an `-hwd` image it is silently wrong (the `HARNESS-BOARD-TEST-CHEATSHEET.md:291` advice "base has no `QEMU_BUILD`" is the opposite of what the code does).
- The Pi `platform-*-support-interface` names the *same* `src/platform-support.cpp` the port compiles; if it were ever linked it would add a second strong `main()` / `_Exit()` to an image that already has one.

- The only live use of `micro-os-plus::platform-support` is the Lyra (`platforms/aarch32-luckfox-lyra/CMakeLists.txt:69`), and the only live uses of `micro-os-plus::platform` are the legacy `qemu-cortex-m*`, `pico2-1cpu`, `2xcortex-m33` and `native` platforms.

[Medium] The live Lyra `platform-support.cpp` is the stale copy; the fixes live only in the (interface-target) Pi copies — `platforms/aarch32-luckfox-lyra/src/platform-`

`support.cpp:99-122` VS `platforms/aarch32-rpi3b/src/platform-support.cpp:107-128,163-206`

CONFIRMED by diff and by `compile_commands.json` (Lyra's file compiles into the `mutex-stress-test` binary; both Pi copies compile through the port). The three functional divergences:

Concern	Lyra (live harness suite)	aarch32/aarch64 Pi copies
Secondary sync	<code>smp_install_boot_threads();</code> <code>smp::start_secondary_cores();</code> — no wait	adds <code>test_wait_secondaries(3000);</code> (<code>rpi3b:109-110</code>)
Semihosting fds / args	<code>os_startup_initialize_args(&argc</code> <code>,&argv) (:112)</code> — issues <code>SYS_GET_CMDLINE</code>	<code>initialise_monitor_handles();</code> (<code>rpi3b:79,92</code>), with a comment that a JTAG run cannot answer <code>SYS_GET_CMDLINE</code> (<code>rpi3b:112-115</code>)
Console mirror	none	<code>os_board_console_mirror()</code> writes stdout to the UART (<code>rpi3b:163-189</code>)

The Lyra harness suite is the one the project intends to promote ("hardware only"), yet it is the copy that lacks the bring-up synchronisation and the UART mirror. I partially agree with the first pass's "never releasing its secondaries": the file *does* call `start_secondary_cores()`, but it omits the `test_wait_secondaries(3000)` handshake the sibling copies added, which is the most likely way a secondary-related hang shows up. The direction of the duplication is inverted from what a reader would assume.

[Medium] AArch64 `_gettimeofday` stub is live but constant; the AArch32 path

instead gets a real weak `gettimeofday` from the port — `platforms/aarch64-rpi-zero-2w/src/platform-support.cpp:157-168` ; `sources/mutex-stress/src/main.cpp:92`

CONFIRMED, **agrees** with the first pass and sharpens it. On AArch64 the stub returns `tv_sec = tv_usec = 0`. Because the platform defines `__ARM_EABI__` (`platform-library.cmake:54`), `busy_wait()` uses `hrclock` and the stub is reached only by `mutex-stress`'s seed (`main.cpp:92`), so it is benign for timing but makes the seed deterministic. The AArch32 binary, by contrast, defines a **weak real** `gettimeofday` from the port (`nm` of the Lyra `mutex-stress-test-hwd` shows `W gettimeofday / W __posix_gettimeofday`), so no `_gettimeofday` stub is needed there and none is present. The divergence is architectural, not a defect, but the file's own comment ("the port does not define [it]") is AArch64-only and is easy to misread as a general rule.

[Medium] Cases are registered per source directory without checking the target;

`file(GLOB)` makes this a silent-coverage trap — `platforms/aarch32-rpi3b/CMakeLists.txt:66-110`, `aarch32-rpi-zero-2w/CMakeLists.txt:55-94`, `aarch64-rpi3b:57-94`, `aarch64-rpi-zero-2w:45-78`, `cortexm-pico2-pizero:31-53`, `cortexm-nucleof411:31-53`, `cortexm-weactf411:31-53`, `cortexm-weactf412:31-53`, `cortexm-pico2-rp2350b-psram:53-89`, `aarch32-luckfox-lyra:35-57`

CONFIRMED by reading, and it is what makes the [High] silent PASS reachable:

```
file (GLOB _test_dirs LIST_DIRECTORIES true "${_port_test_dir}/*")
foreach (_dir IN LISTS _test_dirs)
    ...
    add_test (NAME "${PLATFORM_NAME}-${_app}-hwd"
              COMMAND bash "${_board_hw}" "${_app}" 600)
```

There is no `if (TARGET "${_app}-hwd")` guard (compare `cortexm-pico2/CMakeLists.txt:68` and `cortexm-pico2-rp2350b-psram/CMakeLists.txt:67`, which do guard the `qemu` case). Two consequences: (a) a removed/renamed image leaves a registered case that the runner then reports as PASS (above); (b) because the test list is a configure-time `GLOB`, a test added to the port is invisible until the build tree is re-configured — which the `add_subdirectory` reuse makes easy to forget.

[Medium] Hardware-only `test` action runs zero tests and exits 0 —

`package.json:1173,1206,1325,1368,1413,1446,1480,1728` (and the Pi's `-LE hwd` at `1616,1672,1774`)

CONFIRMED. For every board platform the `test` action is `cd {...} && ctest -V -LE hwd`, while the case set is (almost) entirely labelled `hwd`:

```
"test": "cd {{ properties.buildFolderRelativePath }} && ctest -V -LE hwd"
```

`ctest` with no matching tests prints "No tests were found!!!" and returns 0. So `xpm run test --config cortexm-pico2-pizero-...`, `...-nucleof411-...`, `...-weactf412-...`, `...-luckfox-lyra-...`, and `raspberrypi-pico` all succeed without running a single test. This is intentional filtering, but there is no `if(NOT _board_tests)` or count assertion, and the docs never state it, so it reads as coverage that does not exist.

[Medium] `cortexm-pico2-pizero` claims "QEMU has no Cortex-M33", but its sibling runs the same M33 on QEMU — `platforms/cortexm-pico2-pizero/CMakeLists.txt:4-7`;

`platforms/cortexm-pico2/CMakeLists.txt:72-74`

CONFIRMED. The pizero header says the board "is hardware-only -- QEMU has no Cortex-M33". `cortexm-pico2` is the same RP2350 (2× Cortex-M33) and emits `-qemu` images on `-M mps2-an500 -cpu cortex-m7`. The stated reason is wrong (the reality is that the pizero test list is hwd-only), and the consequence is a coverage inconsistency: `smp-test1`/`sc-test-ko` are emulated on

`pico2` but not on `pizero`. The catalog (`docs/tests/TESTS-CATALOG.md:57-58`) repeats the same "hwd only" claim without explaining the asymmetry.

[Low] `cortexm-pico2` / `rp2350b-psram` qemu cases hard-code the image path instead of

`<TARGET_FILE:...>` — `platforms/cortexm-pico2/CMakeLists.txt:72-74` ;

`platforms/cortexm-pico2-rp2350b-psram/CMakeLists.txt:71-73`

CONFIRMED:

```
"${_qemu}" -M mps2-an500 -cpu cortex-m7  
-kernel "${_port_bin}/test/${_app}-qemu.elf" ...
```

The legacy platforms use `"<TARGET_FILE:${name}>"` (`pico2-1cpu/CMakeLists.txt:43` , `2xcortex-m33/CMakeLists.txt:44`). The hard-coded `...-qemu.elf` silently breaks if the port changes the output name/suffix, and then the [High] silent PASS turns the breakage green.

[Medium] `pico2-1cpu` and `2xcortex-m33` register no CTest labels — `platforms/`

`pico2-1cpu/CMakeLists.txt:39-46` ; `platforms/2xcortex-m33/CMakeLists.txt:37-47`

CONFIRMED: `set_tests_properties (... PROPERTIES TIMEOUT 1200)` sets no `LABELS`. Every other platform labels its cases (`qemu` , `hwd` , `host`). The result is that `ctest -L qemu` (the natural filter, and what a future label-driven action would use) selects none of these even though they are pure QEMU cases. `cortexm-pico2`'s qemu cases are labelled; these are not. Consistency would suggest `LABELS "qemu"`.

[Low] Pinned-toolchain guard is AArch-only, but Cortex-M/native pin toolchains

too — `platforms/aarch32-rpi-zero-2w/cmake/definitions.cmake:26-35` , `aarch32-rpi3b:34-43` ,
`aarch32-luckfox-lyra:37-46` , `aarch64-rpi-zero-2w:22-31` , `aarch64-rpi3b:30-39`

CONFIRMED by grep (the guard exists only in those five files). `package.json` pins `@xpack-dev-tools/arm-none-eabi-gcc 15.2.1-1.1.1` for all Cortex-M configs (`package.json:551` , `681` , `689`) and GCC/Clang versions for native, but `cortexm-*/native` have no equivalent check. The AArch comment describes the exact failure mode ("a bare `cmake` ... silently picks the system compiler"), so the same silent-version mismatch is possible on the other 17 platforms. **Agrees** with, and broadens, the first pass.

[Low] Error messages point to the wrong file for two required variables — `device-`

`qemu-cortexm/CMakeLists.txt:17-20,44-49` ; every `platforms/*/cmake/platform-library.cmake:26-32`

CONFIRMED. The `device` says "Define `xpack_device_compile_definition` ... in `platform/${PLATFORM_NAME}/cmake/definitions.cmake`" (correct) but the linker-script message says "in `platforms/.../dependencies-folders.cmake`" (wrong); the platform library says the same for `xpack_platform_compile_definition`. In fact every platform sets both in `definitions.cmake`. Misleading diagnostics when a new platform is added — the stated path is not where anyone defines it.

[Low] `PLATFORM_NAME` defaults to `unknown` and the "fallback" branch is unreachable

— `tests/cmake/tests-main.cmake:23-25,106-110`

CONFIRMED. If `-D PLATFORM_NAME` is absent, `PLATFORM_NAME=unknown`, and configure dies at line 44 (`include("platforms/unknown/cmake/definitions.cmake")`) long before the fallback `add_subdirectory("../top-bin")` at line 109. So the fallback only ever runs for the real names that don't match the regexes (`qemu-cortex-*`, `raspberrypi-pico`, `nucleo-*`), and the `unknown` default is a dead branch whose error is a missing-file message rather than a clear one.

[Nit] Duplicated project-name prefix — `tests/CMakeLists.txt:27`

CONFIRMED: `project (micro-os-plus-micro-os-plus-iii-${PLATFORM_NAME}-tests ...)` yields `micro-os-plus-micro-os-plus-iii-native-tests`. Almost certainly a carry-over; harmless but visible in every IDE/CTest banner.

[Low] `run-host.sh` and `run-qemu.sh` timeout tables have drifted — `test_smp1/run-`

`host.sh:24-50` ; `test_smp1/run-qemu.sh:35-45,61-81`

CONFIRMED. `run-host.sh` `sd_image_for` handles `flatfs-test` but `timeout_for` does not (falls through to 300); `run-qemu.sh` `sd_image_for` does **not** list `flatfs-test` (only `sd_test`, `smp-mat-sdcard-test`, `smp-num-test`, `smp-pipeline-test`), and its `timeout_for` lacks `flatfs-test` too. Since each runner's comment claims the tables are "the same tables", the divergence should be either removed or the comment corrected.

[Low, SUSPECTED] `run-hw.sh` can abort on a benign OpenOCD `Error:` —

`test_smp1/run-hw.sh:242`

SUSPECTED (no hardware to reproduce). The verdict loop breaks with `rc=5` on `grep -qE '^Error: '`. A `-defer-examine` secondary, or the adapter settling, can emit a transient `Error:` before recovering; that would be reported as "OPENOCD ERROR" even though the run would have succeeded. The `Invalid ACK` case is handled separately (`:239`), suggesting the author knew some `Error:` lines are spurious. Worth a bounded retry rather than an immediate abort.

[Low, SUSPECTED] `OS_USE_OS_APP_CONFIG_H` placement is fragile — `tests/cmake/common-`

`options.cmake:34`

SUSPECTED. The define lives on `micro-os-plus-common-options-interface`, so it reaches an image only if that image links `micro-os-plus::common-options`. The port-builder platforms rely on the port's builder to propagate the suite's `os-app-config.h`; the aarch32-rpi3b header (`CMakeLists.txt:122-128`) documents that when it did not, `rtos-apis` died in `malloc_memory_resource()`. I could not verify from this repo whether every suite on every platform still links it (the aarch64 `compile_commands.json` does show `-DOS_USE_OS_APP_CONFIG_H`), so this is raised as a fragility, not a confirmed regression.

[Low] package.json tidy and CI-coverage drift — package.json:17, 58, 105-125, 193-194

CONFIRMED, several small things:

- `scripts.deep-clean` deletes `... package-json.json` (:17) while `actions.deep-clean` deletes `package-lock.json` (:58); one is wrong.
- `link-deps-all` lists `raspberrypi-pico-cmake-gcc-{debug,release}` twice (:185-186 and :193-194).
- `install` (:105-109) installs only the default config plus the two `aarch32-rpi-zero-2w` configs, and `test` / `test-all` / `test-ci` (:115, 196, 123) all run only `test-aarch32-rpi-zero-2w-cmake`. So the CI entry points exercise one QEMU platform; the other ~21 platforms are never built or run by them.
- `test-raspberrypi-pico-cmake:370-377` and the `nucleo-*` actions inline `cd build/... && ctest -V` rather than calling the config's `test` action, which is why they would try to drive hardware instead of using `-LE hwd`.

Agreement with first pass

- **Luckfox Lyra secondaries — AGREE IN PART.** The live `aarch32-luckfox-lyra/src/platform-support.cpp` does call `smp_install_boot_threads()` + `smp::start_secondary_cores()` (:99-100), so "never releasing" is not literally true of this file; what it lacks is the `test_wait_secondaries(3000)` handshake (`rpi3b:109-110`) and the console mirror. I confirm the file is the stale copy and that the bring-up is the legacy path.
- **Four Pi platforms' unused `platform` / `platform-support` + hard-wired `QEMU_BUILD` / `QEMU linker` — AGREE, with stronger evidence.** Proved via `compile_commands.json` that the file the interface targets name is compiled by the port, and that no image links the targets. Added the "landmine" framing and the aarch32 family (not just the four Pi platforms) as affected.
- **Pinned-toolchain guard only in aarch32/aarch64 — AGREE,** and broaden: the Cortex-M and native configs also pin exact toolchains without a guard.
- **`ENABLE_HW_TESTS=ON` unused — AGREE.** `package.json:1725` passes it; no `.cmake` reads it; `docs/tests/HARNESS-BOARD-TEST-CHEATSHEET.md:312-325, 457` documents it as the prescribed mechanism anyway.
- **Legacy `qemu-cortex` cases without labels/timeouts — AGREE,** and added the silent-PASS consequence when their image is absent, plus their unprefixed case names.
- **AArch64 `_gettimeofday` stub constant — AGREE,** with the nuance that it is seed-only (because `__ARM_EABI__` routes `busy_wait` to `hrclock`) and AArch64-only (AArch32 gets a weak real `gettimeofday` from the port).

Documentation drift

- `tests/README.md` is the most stale document in scope. Its platform list (:19-37) predates the entire SMP work: it has no `aarch32` / `aarch64` / `Luckfox` / `cortexm-pico2` / `2xcortex-m33` /

`pico2-1cpu / nucleof411 / weactf*`. It says `qemu-cortex-m0` runs "the M0 code" (`:29-31`) while `TESTS-CATALOG.md:65` correctly notes the platform passes `--cpu cortex-m3`. It says the Cortex-M toolchain is "arm-none-eabi-gcc 14" (`:56`) while `package.json` pins 15.2.1. It links `.github/workflows/ci.yml / test-all.yml` (`:10,15`) but `.github/` does not exist in the repo. Its `while (true)` recipes (`:64,69`) use `~/Work/micro-os-plus-iii/micro-os-plus-iii.git/tests`, a path that no longer exists.

- **Cheatsheet describes an `ENABLE_HW_TESTS` option that no platform implements** (`docs/tests/HARNESS-BOARD-TEST-CHEATSHEET.md:291,312-325,457`) and asserts the platform base should carry "no `QEMU_BUILD`" (`:291`) while the four Pi `platform-library.cmake` files define `QEMU_BUILD` unconditionally (`aarch64-rpi-zero-2w:53` etc.). The cheatsheet's own template at `:108` includes `QEMU_BUILD` in the base.
- **`AARCH32-RPI-ZERO-2W-TESTS.md` references a design that no longer exists** (`test_smpl/common/`, `run-qemu-aarch32.sh`, `run-hw-aarch32.sh`, `stdio-shim.hpp`, `test-console.hpp` at `:6,355,400,560,587,618`), which its own banner (`:6`) admits. Historically useful, currently misleading as a "how to add a platform" guide.
- **`docs/tests/TESTS-CATALOG.md` is largely accurate** and is the best-kept document; the only drift I found is the pizero "hwd only" rationale (above) and the unsupported blanket statement that "every platform has a `-cmake-gcc-debug` and `-cmake-gcc-release` configuration" (`:74`) — the native platform's canonical configs also include `-cmake-sys-*`, `-cmake-gccNN-*`, `-cmake-clangNN-*`, which the very next line acknowledges.
- **`sources/*/CMakeLists.txt` consumption comment is wrong:** `add_subdirectory("tests/blinky")` vs the real `tests/sources/blinky` (`sources/blinky/CMakeLists.txt:16`, and the sibling suites).
- **Orphaned reference:** the `aarch32/aarch64` `platform-support.cpp` comments describe the port's `test/<board>/tests.cmake` harness-suite interface as the thing that links them; for the four Pi platforms the interface target in this repo is instead the dead one (finding above), so the comment and the code tell different stories.

Good practices

- **No double registration.** The generated `CTestTestfile.cmake` for the Pi platforms contains 30 distinct `add_test` names (15 qemu + 15 hwd) with no duplicates: the port's builder registers no CTest cases of its own (its `CTestTestfile.cmake` body is empty), so the platform's loop is the single registration point. (`uniq -d` over the names is empty.)
- **The runner verdict protocol is uniform and documented.** `RESULT: PASS|SKIP|FAIL`, the NO-RESULT fallback, and the 124-timeout branch are the same in all three runners, and `hw_result.hpp` explains the "print then stop" contract. The `run-hw.sh` refusal to run a suite (`:280-286`), the per-core semihosting enable (`:207-210`), and the `__smp_spin` zeroing before resume (`:144-153`) are careful, correct choices for a no-SRST debugger run.
- **`board-contract.cpp` turns missing board facts into compile errors** (`test_smpl/src/board-contract.cpp:55-125`), with a clear rationale and no defaults in the shared tree. This is exactly the right mechanism and its absence is what let the earlier silent board-fact bugs happen.

- **The port-builder reuse pattern** (glob the port's test dirs, register `<platform>-<app>-<variant>`, let the port own the tests) keeps test lists in one place. When combined with a `TARGET` check (as `cortexm-pico2` does for qemu) it is robust; the fix for the [High] finding is to extend that check to every variant and to the hwd loops.
- `hw_result.hpp` and `board-contract.cpp` are shared from the kernel repo rather than copied per port, and the root `CMakeLists.txt:206-230` explains the opt-in target and the link-order reason for naming the file directly. That is the right answer to the duplication the code still carries in `platform-support.cpp`.

First pass — detailed findings

µOS++ Ille SMP kernel — read-only code review

Scope

Reviewed, read-only, under `/home/dan/Work`:

- Kernel sources `micro-os-plus-iii-smp.git/src/` (rtos, memory, semihosting, posix-io, startup, utils, libcpp).
- Kernel headers `micro-os-plus-iii-smp.git/include/cmsis-plus/` (rtos, memory, utils, posix-io, diag).
- Harness test sources `micro-os-plus-iii-smp.git/tests/sources/` and shared runners `micro-os-plus-iii-smp.git/test_smpl/`.

Context used to judge the kernel (not modified): the port contract `port/smp-common/cmsis-plus/rtos/port/os-decls.h` declares the cross-core lock (`_smp_klock`, per-core `lock_state[]`, `_port_ctx_pending[]`); the lock and context-switch implementations live in the architecture repos (`micro-os-plus-iii-aarch64`, `-aarch32`, `-posix-arch`), so a few observations about lock/memory-ordering are marked as depending on those ports. The current working tree was reviewed as-is (it already contains uncommitted condvar/detach changes over commit `55000fe`).

No files were modified except this report.

Summary

The kernel core is in good shape: the interrupt/scheduler RAIL critical sections correctly acquire the recursive per-core kernel lock, the deferred-publish/claim pattern in the ports is documented at length, and the SMP-sensitive paths (`join()`, `kill()`, idle reaper, ready-list picker) already carry fixes for earlier

life-cycle races. The most important remaining risks are **cross-core non-atomic reads of `current_thread_[]`, `state_` and `stack_ptr`** (technically a C++ data race) and **memory resources that ignore the requested alignment**, including the absence of over-aligned `operator new / delete` overloads. Beyond those, most findings are concrete but lower-severity defects in the semihosting syscall shim, the POSIX fd table, the first-fit/block-pool allocators, and the list/iterator helpers, plus shell-runner robustness issues.

Severity tally: 0 Critical, 3 High, 11 Medium, 9 Low/Nit (23 findings).

Findings

[HIGH] Cross-core non-atomic reads of the scheduler's shared state — `src/rtos/os-thread.cpp:1104, 1460`; `src/rtos/os-idle.cpp:91`; `include/cmsis-plus/rtos/os-sched.h:59`

```
// os-thread.cpp
for (unsigned c = 0; c < OS_NCPU; ++c)
    if (scheduler::current_thread_[c] == this) { still_running = true; break; }
...
bool busy = (context_.port_.stack_ptr == nullptr);
for (unsigned c = 0; c < OS_NCPU; ++c)
    if (scheduler::current_thread_[c] == this) { busy = true; ... }
```

`current_thread_` is declared `thread* volatile current_thread_[OS_NCPU]` and `state_` is `state_t volatile` (`os-thread.h:1699`); `context_.port_.stack_ptr` is written by the ports with `__atomic_store_n(..., __ATOMIC_RELEASE)` (see `posix-arch/src/rtos/os-core.cpp:315,354`). `join()`, `kill()` and the idle reaper read all of these from *other* cores without the kernel lock (the lock is dropped for the wait/yield loops) and without atomics or a matching acquire. `volatile` orders only the compiler's access to that object, not CPU visibility or ordering.

Impact: a formal C++ data race (UB) on the hot scheduler path; on weakly-ordered cores the `stack_ptr` / `current_thread_` update can be observed late, extending the window in which `join()` / `kill()` may decide a thread is quiescent while another core is still restoring or about to run it. The busy-wait loops make a wrong decision unlikely in practice, so this is **confirmed UB / suspected rare misbehaviour**, not a demonstrated failure.

Suggestion: make `current_thread_[]`, `state_` and `stack_ptr` `std::atomic` (or use `__atomic_load_n(..., __ATOMIC_ACQUIRE)` in these three readers), or read them under the kernel lock in a short retry loop.

[HIGH] Memory resources ignore the requested alignment — include/cmsis-plus/memory/malloc.h:242, 279; src/libcpp/new.cpp (no aligned overloads)

```
// malloc_memory_resource::do_allocate
// Ignore alignment for now.
void* mem = std::malloc (bytes);
...
// new_delete_memory_resource::do_allocate
// Ignore alignment for now.
void* mem = ::operator new (bytes);
```

`memory_resource::allocate(bytes, alignment)` passes an alignment that both resources discard. `operator new` / `operator delete` in `src/libcpp/new.cpp` also define only the `size_t` and `nothrow` forms — there is no `operator new(std::size_t, std::align_val_t)` / `operator delete(void*, std::align_val_t)`.

Impact (confirmed): any over-aligned type (`alignof(T) > alignof(std::max_align_t)`, e.g. a 64-byte cache-line struct, `alignas(64)` buffers, SIMD types) allocated through these resources receives under-aligned storage → UB and possible faults. On the host it silently succeeds because glibc returns 16-byte alignment, so it will not show up in the POSIX tests.

Suggestion: honour `alignment` (over-allocate + store the offset, or `std::aligned_alloc` / `posix_memalign` on host, aligned `operator new` in C++), and add the `align_val_t` overloads to `new.cpp`.

[HIGH] `block_pool::internal_construct_` has an inverted failure assert (and overstates the arena) — src/memory/block-pool.cpp:167

```
res = std::align (alignof (void*), blocks * block_size_bytes_,
                 pool_addr_, align_sz);
// std::align() will fail if it cannot fit the adjusted block size.
if (res != nullptr)           // <-- should be "if (res == nullptr)"
{
    assert (res != nullptr);
}
total_bytes_ = blocks_ * block_size_bytes_;
```

On failure (`res == nullptr`) nothing is reported; on success the assert is trivially true. `first_fit_top::internal_construct_` does this correctly (`first-fit-top.cpp:57`, `if (res == nullptr)`), which shows the intended pattern. `total_bytes_` is then set to the un-aligned requested size even if the arena cannot actually hold it.

Impact (confirmed): an undersized/misaligned pool silently advertises more capacity than exists; a later `do_allocate` writes block-link words past the arena. Debug builds will not catch it.

Suggestion: assert on `res == nullptr` and compute `total_bytes_` from the adjusted `align_sz`.

[MEDIUM] Semihosting fd table is unsynchronised and `newslot()` does not reserve — `src/semihosting/c-syscalls-semihosting.cpp:140, 300, 358`

```
static int __semihosting_newslot (void) { ... return i; } // does not mark it used
...
int fd = __semihosting_newslot (); // pick
...
int fh = call_host (SEMIHOSTING_SYS_OPEN, block); // host call
if (fh >= 0) { openfiles[fd].handle = fh; ... } // mark much later
```

`openfiles[]` is a plain global array; `__semihosting_findslot`, `__semihosting_newslot`, `__posix_open` and `__posix_close` all mutate it without any lock. The slot is claimed only after the (slow) host call, so two threads/cores racing `open()` can be handed the same `fd`. The same global table and the single semihosting debug channel are also shared across cores.

Impact (confirmed for single-threaded use; **suspected** under concurrent open/close): two different `io` objects share one descriptor slot, one handle leaks, or a close recycles a slot still in use.

Suggestion: guard the fd table with a critical section (or a dedicated spinlock) and reserve the slot before the host call, releasing it on failure.

[MEDIUM] `__posix_stat` produces an invalid `st_mode` (`S_IFREG` OR `S_IFCHR`) — `src/semihosting/c-syscalls-semihosting.cpp:556, 268`

```
// __posix_stat
buf->st_mode |= S_IFREG | S_IREAD;
int res = __semihosting_stat (fd, buf);
...
// __semihosting_stat
st->st_mode |= S_IFCHR;
```

`S_IFREG` (0x8000) and `S_IFCHR` (0x2000) share the `S_IFMT` mask, so the result is 0xA100 — neither a regular file nor a character device. `S_ISREG()` / `S_ISCHR()` both fail.

Impact (confirmed): code that branches on the file type (e.g. FatFs/posix users, `ls`-like diagnostics) misclassifies every file opened via `stat()`.

Suggestion: assign rather than OR, or make `__semihosting_stat` set `S_IFCHR` only when the caller did not already set a type.

[MEDIUM] `file_descriptors_manager::deallocate` / `socket` dereference a possibly-null slot — `src/posix-io/file-descriptors-manager.cpp:197, 211`

```
descriptors_array__[fildes]->clear_file_descriptor (); // deallocate, no null check
...
auto* const io = descriptors_array__[fildes];           // socket
if (io->get_type () != ...)                             // io may be nullptr
```

`io(fildes)` deliberately returns `nullptr` for an empty slot (line 90), but `deallocate()` and `socket()` index the array directly and dereference. `assign()` (line 154) can also install an `io` over an occupied slot without clearing the previous owner.

Impact (confirmed): a double `close()` / `close()` of a never-opened descriptor, or `getsockopt`-style calls on a stale fd, crash on a null dereference instead of returning `EBADF`. A re-`assign` leaks the previous `io`'s descriptor bookkeeping.

Suggestion: null-check the slot in `deallocate()` / `socket()` and return `EBADF`; in `assign()` reject an already-occupied slot (or clear it first).

[MEDIUM] CMSIS timeout conversion overflows 32 bits before widening — `src/rtos/os-c-wrapper.cpp:3755` (and 3804, 3883, 4067, 4178, 4371, 4661, 4759, 4897, 5060)

```
result_t res = sysclock.sleep_for (
    clock_systick::ticks_cast ((uint64_t)(millisec * 1000u)));
```

`millisec` is `uint32_t`; `millisec * 1000u` wraps modulo 2^{32} before the cast to `uint64_t`, for any `millisec > 4_294_967` ms (≈ 71.6 min). `osWaitForever` is special-cased in most callers, but a large finite timeout is not.

Impact (confirmed): timeouts above ~ 71 min silently become a small/zero timeout — the call returns early (`osEventTimeout`) instead of blocking as asked.

Suggestion: `(uint64_t)millisec * 1000u`, or better convert via a helper that takes `uint64_t` throughout.

[MEDIUM] `condition_variable::timed_wait` decides ETIMEDOUT from the wall clock after re-locking — `src/rtos/os-condvar.cpp:771, 811`

```
clock::timestamp_t timeout_timestamp = clk->steady_now () + timeout;
...
res = mutex.lock ();
if (res == result::ok && clk->steady_now () >= timeout_timestamp)
    res = ETIMEDOUT;
```

The timeout is inferred from the clock *after* the mutex is re-acquired, not from which event (timer expiry vs `signal` / `broadcast`) actually woke the thread. A thread that was signalled just after its deadline — but that re-acquires the mutex before the timer node handler runs — is reported as `ETIMEDOUT`. Also `steady_now() + timeout` can wrap `timestamp_t` with no check.

Impact (confirmed): spurious `ETIMEDOUT` even though the condition was signalled (POSIX allows either outcome only when the timeout genuinely expired first), and wraparound for huge timeouts. Condition-variable wait loops that treat `ETIMEDOUT` as fatal will misbehave.

Suggestion: remember whether the timeout node's `action()` ran (or whether the CV link was cleared by signal/broadcast) and only report `ETIMEDOUT` in the timer case; saturate the deadline.

[MEDIUM] `int` truncation of a `size_t` difference in the allocators — `src/memory/first-fit-top.cpp:143; src/memory/lifo.cpp:91`

```
int rem = static_cast<int> (chunk->size - alloc_size);
if (rem >= 0) { ... }
```

When `chunk->size < alloc_size` the subtraction underflows to a huge `size_t`; converting that to `int` is implementation-defined (pre-C++20) and only "works" because the low 32 bits happen to be negative on the tested ABIs. On a 64-bit `size_t` this is fragile and non-portable, and it silently relies on two's-complement wrapping.

Impact (confirmed portability/UB risk; correct on GCC/Clang targets): the "does it fit" test can be mis-evaluated by a conforming compiler with different conversion semantics.

Suggestion: compare directly with `chunk->size >= alloc_size` and use `std::size_t rem = chunk->size - alloc_size;`, or cast through `std::ptrdiff_t` after the guard.

[MEDIUM] `thread::detach()` rejects parentless threads and is fragile on SMP — `src/rtos/os-thread.cpp:1004`

```
if (state_ == state::destroyed || parent_ == nullptr)
{
    instrumentation::thread::detach_retval (this, EINVAL);
    return EINVAL;
}
child_links_.unlink ();
scheduler::top_threads_list_.link (*this);
parent_ = nullptr;
```

Top-level threads are created with `parent_ == nullptr` (`internal_construct_`, line 567-575) and appear on `top_threads_list_`; the new code therefore returns `EINVAL` for exactly those threads, even though they are joinable. It also performs the relink under `interrupts::critical_section` only, without any state/ `parent_` recheck after the lock is dropped (the check is outside the section), so a concurrent `internal_exit_()` can interleave.

Impact (confirmed behaviour change): the previous stub returned `result::ok`; code that detaches a top-level thread now fails with `EINVAL`. On SMP a detach racing a termination can leave the thread on `top_threads_list_` after destruction (**suspected** dangling node).

Suggestion: treat an already-`top_threads_list_` thread as success, move the whole test/relink under one critical section, and re-check `state_ == terminated/destroyed`.

[MEDIUM] `waiting_threads_list::link` does not initialise a first-use list, unlike its siblings — `src/rtos/internal/os-lists.cpp:193`

```
void waiting_threads_list::link (waiting_thread_node& node)
{
    thread::priority_t prio = node.thread_>priority ();
    waiting_thread_node* after = static_cast<waiting_thread_node*> (
        const_cast<utils::static_double_list_links*> (tail ()));
    if (empty ()) { ... }
```

`ready_threads_list::link` (line 47) and `terminated_threads_list::link` (line 535) both begin with `if (head_.prev () == nullptr) clear ();`. `waiting_threads_list` inherits `utils::double_list`, whose constructor clears the list, so constructed objects are fine — but a BSS-zero list (the C API places some objects in user memory) would give `tail() == nullptr` and `insert_after()` would dereference null (`lists.cpp:136`).

Impact (confirmed inconsistency; latent null-deref): the invariant "`insert_after` is always given linked neighbours" is only maintained by construction, not enforced where its siblings enforce it.

Suggestion: add the same first-use `clear()` guard, or document and assert `!uninitialized()`.

[MEDIUM] `double_list_iterator` post-increment/decrement use member functions as data — `include/cmsis-plus/utils/lists.h:920, 928, 937`

```
node_ = static_cast<iterator_pointer> (node_>next);    // operator++(int)
...
node_ = static_cast<iterator_pointer> (node_>prev);    // operator--()
...
node_ = static_cast<iterator_pointer> (node_>prev);    // operator--(int)
```

`static_double_list_links` declares `next_` / `prev_` data members and `next()` / `prev()` functions; `node_>next` names the member function, and casting a pointer-to-member-function to `N*` is ill-formed. The prefix forms (lines 911, 928) correctly call `next()` / `prev()`. These templates are only instantiated for `waiting_threads_list`, whose `begin()` is not used by the kernel today, so the defect is latent.

Impact (confirmed latent compile break / wrong code if instantiated): anyone using post-increment/decrement on a `waiting_threads_list` iterator fails to compile (or is silently mis-handled if it ever compiled).

Suggestion: `node_ = static_cast<iterator_pointer> (node_>next ());` etc.

[MEDIUM] Test-runner log greps can declare false verdicts and leak the tee process — `test_smpl/run-hw.sh:239, 242`; `test_smpl/run-host.sh:58`; `test_smpl/run-qemu.sh:89`

```
if grep -qE 'Invalid ACK|Polling failed|Examination failed' "$log"; then rc=6; break; fi
if grep -qE '^Error: ' "$log"; then rc=5; break; fi
...
"$OPENOCD" "${args[@]}" > >(tee "$log") 2>&1 &
local ocd=$!
...
kill "$ocd"; wait "$ocd"
```

`^Error:` also matches benign OpenOCD errors (a missing optional target, a flash probe warning) that do not abort the run, turning a passing test into `OPENOCD ERROR`. The `kill` / `wait` pair reaps OpenOCD but not the `tee` process substitution, so the log can still be written after the verdict is read. In `run-host.sh:58` and `run-qemu.sh:89`, `exit 2` on the first non-executable `*-host` / `*-qemu.bin` entry aborts the whole suite instead of skipping it (a stray non-executable file is enough).

Impact (confirmed robustness): flaky CI verdicts and occasional truncated log reads; a single stray file aborts a multi-test run.

Suggestion: match only fatal OpenOCD diagnostics (or gate on the process having exited first), wait on the `tee` PID as well, and `continue` / skip rather than `exit` per-entry.

[MEDIUM] Memory allocators are only safe when externally locked — `src/memory/first-fit-top.cpp:120`; `src/memory/block-pool.cpp:48`; `src/memory/lifo.cpp:70`

`first_fit_top::do_allocate/do_deallocate`, `block_pool::do_allocate/do_deallocate` and `lifo::do_allocate` mutate free lists/statistics with no internal critical section. They are safe only because the paths that are actually used (`operator new` in `src/libcpp/new.cpp:145`, and `allocator_stateless_polymorphic_synchronized` in `os-memory.h:1575`) wrap calls in a scheduler/kernel lock.

Impact (suspected misuse hazard): any direct `resource.allocate()/deallocate()` from two cores (the `pmr::memory_resource` interface is public) corrupts the free list. The classes' own docs do not warn about this for `first_fit_top`.

Suggestion: either document "not thread safe; lock externally" on `first_fit_top`/`lifo`/`block_pool`, or add an internal lock.

[LOW] `first_fit_top`/`memory_pool` use relational comparison on unrelated pointers — `src/memory/first-fit-top.cpp:259`; `src/rtos/os-mempool.cpp:826`

```
if ((addr < arena_addr_) || (addr > (static_cast<char*> (arena_addr_) + total_bytes_)))
```

Relational comparison of `void*`/`char*` values that are not part of the same array is unspecified/UB in C++. Also the upper bound uses `>` where `>=` is the correct exclusive limit (the chunk metadata at `arena_end - chunk_offset` is not a valid payload).

Impact (confirmed portability nit; benign on flat embedded maps).

Suggestion: compare `reinterpret_cast<uintptr_t>` values and use `>=` for the high end.

[LOW] Semihosting `lseek` narrows a 64-bit offset to `int` — `src/semihosting/c-syscalls-semihosting.cpp:188, 210, 246`

```
static int __semihosting_lseek (int fd, int ptr, int dir) ...
ptr = pfd->pos + ptr;
...
pfd->pos = ptr;
```

`__posix_lseek` passes an `off_t`, and `pfd->pos` is `int`. Files ≥ 2 GiB cannot be addressed, and `pos + ptr` can overflow signed `int` (UB).

Impact (confirmed on large files; Fine for small embedded images).

Suggestion: keep the semihosting `fh`/position in `off_t` (or `long long`) and range-check before the host `SYS SEEK` call.

[LOW] `__posix_system` has a dead `exit_code` and a dubious shift loop — `src/semihosting/c-syscalls-semihosting.cpp:606`

```
int e = __semihosting_checkerror (call_host (SEMIHOSTING_SYS_SYSTEM, block));
if ((e >= 0) && (e < 256))
{
    int exit_code;
    for (exit_code = e; (e != 0) && (WEXITSTATUS (e) != exit_code); e <<= 1) { continue; }
}
return e;
```

The loop computes nothing observable (`exit_code` is unused, `e` may be shifted out), so the intended exit-status normalisation is a no-op.

Impact (confirmed): `system()` returns the raw host value despite the comment claiming conversion.

Suggestion: implement the newlib exit-status encoding or delete the dead loop and use `WEXITSTATUS`.

[LOW] `__posix_getcwd` ignores `size` semantics — `src/semihosting/c-syscalls-semihosting.cpp:669`

```
char* __posix_getcwd (char* buf, size_t size)
{
    strncpy (buf, "/tmp", size);
    return buf;
}
```

If `size < 5` the result is not NUL-terminated; `strncpy(..., 0)` writes nothing. POSIX requires `NULL` + `ERANGE` when the buffer is too small. `buf == nullptr` also faults.

Impact (confirmed): caller buffer overrun / non-terminated string for small buffers.

Suggestion: `if (buf == nullptr || size < 5) { errno = ERANGE; return nullptr; }` then copy.

[LOW] `file_descriptors_manager::valid()` returns true for in-range but unallocated fds — `src/posix-io/file-descriptors-manager.cpp:106`

```
bool file_descriptors_manager::valid (int fildes)
{
    if ((fildes < 0) || (static_cast<std::size_t> (fildes) >= size_)) return false;
    return true;
}
```

It only range-checks, unlike `io(fildes)` which also checks the slot and `descriptors_array__ != nullptr`. A caller using `valid()` as "is this an open descriptor" gets the wrong answer.

Impact (confirmed semantic bug): stale/never-opened fds pass a validity test.

Suggestion: have `valid()` test `descriptors_array__ != nullptr && descriptors_array__[fildes] != nullptr`.

[LOW] `osMessageGet` leaves `osEvent` fields uninitialised — `src/rtos/os-c-wrapper.cpp:4721`

```
osEvent event;
...
event.status = ...;    // value.v / value.p / def set only on some paths
return event;
```

`osEvent.def` is never assigned and `value` is only written on the message path; the non-message paths return a partially-uninitialised aggregate to the caller.

Impact (confirmed): callers reading `event.def` / `event.value.p` on a timeout or error get indeterminate data.

Suggestion: value-initialise `osEvent event{}`; and set `event.def` / `value` as the CMSIS API specifies.

[LOW] Fallback to a null idle thread in the SMP picker — `src/rtos/os-core.cpp:623`

```
else
{
    scheduler::current_thread[cpu] = scheduler::os_idle_thread_core[cpu];
    if (scheduler::os_idle_thread_core[cpu] != nullptr)
        scheduler::os_idle_thread_core[cpu]->state_ = thread::state::running;
}
instrumentation::thread::active (scheduler::current_thread[cpu]);
```

During the boot window a secondary core can have no registered idle thread; the picker then installs `nullptr` as the current thread and passes it to instrumentation. The ports abort on a null `new_thread` after this call (e.g. `aarch64/src/rtos/os-core.cpp`), and `instrumentation::thread::active(nullptr)` may dereference.

Impact (suspected, boot-window only): null dereference if instrumentation is enabled before the secondary idle thread is registered.

Suggestion: guard `instrumentation::thread::active(...)` with a null check, or keep the per-core fake/boot thread current until the idle thread is registered.

[LOW] `os-c-wrapper` comment contradicts the `try_receive` result contract — `src/rtos/os-c-wrapper.cpp:4746`

```
.resume ... try_receive (... , nullptr);  
// result::event_message when message;  
// result::ok when no message
```

`message_queue::try_receive` actually returns `result::ok` when a message was received and `EWOULDDBLOCK` when the queue is empty (`src/rtos/os-mqueue.cpp:1513-1518`). The code below it maps `result::ok` → `osEventMessage` and `EWOULDDBLOCK` → `osOK`, which is correct; only the comment is inverted. Documentation-only, but the same comments in `osMessagePut` / `osMessageGet` will mislead the next reader.

Impact (confirmed doc bug; no runtime effect).

Suggestion: correct the comment to match `os-mqueue.cpp`.

[NIT] `run-host.sh` / `run-qemu.sh` collect `results[]` but never use it — `test_smpl/run-host.sh:53`, `test_smpl/run-qemu.sh:84`

`declare -a results=()` is appended to for every verdict but never printed or returned. Harmless, but dead state that suggests a lost summary feature (a JUnit/machine report).

Suggestion: print the array in the summary, or drop it.

Good practices

- RAIL critical sections (`interrupts::critical_section`, `scheduler::critical_section`, `uncritical_section`) are used consistently, and the recursive per-core kernel lock in the ports makes nesting safe; the kernel's own `internal_switch_threads` / `internal_link_node` document their locking preconditions.
- The SMP life-cycle paths carry explicit, well-commented fixes: the `join()` reclaimer waits until no CPU has the target current (`os-thread.cpp:1099-1121`), `kill()` has the analogous gate with `port_smp_ipi()` (`os-thread.cpp:1442-1498`), and the idle reaper applies the same "off every CPU and context saved" rule (`os-idle.cpp:83-100`).
- The deferred publish/claim protocol is documented in the kernel comment at `os-core.cpp:589-592` and implemented with release stores in the ports; the ordering rationale for releasing the kernel lock word last is spelled out and was clearly learned from real deadlocks.
- The board contract file (`test_smpl/src/board-contract.cpp`) turns missing per-board facts into compile errors — an excellent pattern that prevented several silent defaults.
- Test runners give explicit per-test wall-clock budgets and classify PASS/SKIP/FAIL/TIMEOUT/NO-RESULT, and `run-hw.sh` refuses to run a suite because a power cycle is required between tests (`run-hw.sh:280`).

- `test_smpl/include/hw_result.hpp` centralises the hardware PASS/FAIL handshake so QEMU and silicon runs share one verdict format.

µOS++ Ille AArch32 / AArch64 port review

Scope

Read-only review of the two architecture ports and their board/test trees:

- `micro-os-plus-iii-aarch32.git`: `include/`, `src/` (`context_switch.cpp`, `exception_handler.cpp`, `handlers.cpp`, `rtos/os-core.cpp`, `semihosting-exit.cpp`, `smp_secondary.cpp`) and `test/boards/{rpi-zero-2w, luckfox-lyra}/**` (`startup.S`, `smp.cpp`, `mmu.cpp`, `timer_arm.cpp`, `uart.hpp`, `gic.hpp`, `rtos/port_isr.cpp`, `port_sys.cpp`, `USB/DWC2`).
- `micro-os-plus-iii-aarch64.git`: same top-level layout plus `test/boards/rpi-zero-2w/**`.

Supporting facts were cross-checked against the kernel (`micro-os-plus-iii-smp.git`: `src/rtos/os-core.cpp`, `os-sched.h`, `port/smp-common/.../os-decls.h`) and the BCM2837 device headers. No files were modified; this report is the only artifact written.

Summary

The two ports share a genuinely careful SMP design: a single recursive LDREX/STREX (AArch32) / LDAXR/STLXR (AArch64) kernel lock, a deferred "publish" so a switched-out `stack_ptr` is only visible after SP has left the outgoing stack, a claim-by-nulling protocol in `internal_switch_threads`, a tripwire that validates the inbound frame, and a fault console that deliberately avoids the semihosting trap. The 64-bit and 32-bit variants are structurally parallel and the divergences are mostly intentional ISA splits.

The most consequential defect found is a **timing inconsistency on the Raspberry Pi AArch32 port**: the kernel's high-resolution clock reads the raw `CNTFRQ` (`timer_arm::get_freq()`) even though the board's own code documents that this value is wrong and calibrates the real rate into `frequency()`; the two must be read from the same place. Other real issues are the AArch64 MMU leaving TTBR1 walks enabled with `T1SZ=0`, the USB gadget stack compiling only at `-O0`, a misleading `OS_HAS_INTERRUPTS_STACK` declaration, silent secondary-release timeouts, and several smaller attribute/type divergences. No remotely-triggerable memory-corruption or data-race defect was confirmed in the scheduler lock or the publish protocol itself.

Findings

[High] High-res clock uses the known-wrong CNTFRQ while the tick uses the calibrated rate — `include/cmsis-plus/rtos/port/os-inlines.h:309` (and `:315`)

```
clock_highres::input_clock_frequency_hz (void)
{
    return timer_arm::get_freq();
}
...
clock_highres::cycles_per_tick (void)
{
    return timer_arm::get_freq() / 1000;
}
```

On the Pi Zero 2W the board explicitly refuses to trust `CNTFRQ` and measures the counter instead (`test/boards/rpi-zero-2w/src/timer_arm.cpp:29-59`, `frequency() / period_cycles()` use the calibrated `g_freq`), yet the entire `clock_highres` path uses `timer_arm::get_freq()`, which is the raw `mrc p15,0, ..., c14, c0, 0` read (`include/timer_arm.hpp:41-45`). The file's own comment says `CNTFRQ` "reports 19.2 MHz while the physical generic-timer counter actually increments at ~1 MHz". Consequently `std::chrono / hrclock` durations derived from `input_clock_frequency_hz()` are wrong by that ratio. Confirmed defect (latent until an app uses the high-res clock). Fix: have `clock_highres::input_clock_frequency_hz() / cycles_per_tick()` call `timer_arm::frequency()` (the calibrated value), or make the board override `get_freq()`.

[Medium] AArch64 MMU leaves TTBR1 enabled with T1SZ=0 — `test/boards/rpi-zero-2w/src/mmu.cpp:139`

```
std::uint64_t tcr =
    (30ULL << 0) | // T0SZ
    ...
    (2ULL << 32); // IPS = 40-bit
```

`TCR_EL1` is written once with `T1SZ = 0` and `EPD1 = 0`, so TTBR1 table walks remain enabled over the whole 64-bit space, overlapping the TTBR0 region; `TTBR1_EL1` itself is never programmed. All current code uses low VAs so TTBR1 is not selected, but any negative/interior 64-bit pointer (a common consequence of an ABI bug) turns into an unpredictable walk through an uninitialised TTBR1 instead of a clean translation fault. Suspicion/robustness (not observed). Fix: set `EPD1 = 1` (and `T1SZ`), or program TTBR1.

[Medium] DWC2 gadget stack only enumerates at `-O0` — `test/luckfox-lyra/`

`tests.cmake:162`

```
# This is not a preference and not a workaround we could drop: the source under
# usb/ is BYTE-IDENTICAL ... but only at -O0.
...
PROPERTIES COMPILE_OPTIONS "-O0"
```

The comment itself records that at `-O2` the EP0/descriptor path is "rewritten into something the hardware will not enumerate with". That is the signature of optimisation-sensitive code (missing `volatile`, missing barrier, or relying on statement ordering/timing) rather than a toolchain preference; the workaround hides a latent compiler-dependence. Confirmed as a build-fragility defect (the issue is documented, the root cause is not fixed). Fix: identify the `volatile` / `asm` ordering the `-O2` build violates; do not rely on `-O0`.

[Medium] AArch32 SYS_EXIT passes a bare reason word where the Angel/OpenOCD ABI is a `{reason, status}` block — `include/semihosting.hpp:108`

```
register const int* r1 asm ("r1") = &code;
asm volatile ( UOS_SEMIHOST_TRAP ... );
```

The hardware (non-`QEMU_BUILD`) AArch32 path hands the debugger a pointer to a single `int` (the reason). The AArch64 sibling correctly passes a two-field block (`include/semihosting.hpp:67-73` in the aarch64 repo: `volatile uint64_t block[2] = { reason, status };`). Implementations that follow the modern `{reason, subcode}` parameter block read the second word from adjacent stack memory, so the reported exit status is garbage; only the printed `RESULT:` line saves the run. Suspicion (ABI is debugger/version-dependent), but the asymmetry with the AArch64 port is confirmed. Fix: pass a two-word block, matching AArch64.

[Medium] Secondary-core release timeout is silently ignored — `test/boards/rpi-zero-2w/src/smp.cpp:63` (AArch32) and `test/boards/rpi-zero-2w/src/smp.cpp:68` (AArch64)

```
for (unsigned spin = 0; spin < 200000000u; ++spin) {
    __asm__ volatile("dmb" ::: "memory");
    if (_port_core_alive[core])
        break;
    ...
}
```

If a secondary never ticks (bad mail-box write, missing timer route, cache coherency handoff failure), `release_one()` gives up and `start_secondary_cores()` releases the next core anyway, with no error surfaced. The failure then appears later as a hang or a wrong `g_core_stage` in an unrelated test. The AArch64 version at least prints a `DEBUG_BOOT`-only message (`test/boards/rpi-zero-2w/src/`

`smp.cpp:75-78`). Confirmed. Fix: make the timeout a hard error reported through the board console (and the semihosting result).

[Medium] AArch64 port ignores `OS_SYSTICK_DIV` that AArch32 supports — `test/boards/rpi-zero-2w/src/rtos/port_isr.cpp:65`

```
uint32_t freq = timer_arm::get_freq();
timer_arm::set_tval(freq / 1000);
```

The AArch32 port re-arms with `timer_arm::period_cycles()` and phases the global tick by `OS_SYSTICK_DIV` (`aarch32 .../port_isr.cpp:88-101`, `include/timer_arm.hpp:25-27`). The AArch64 port hard-codes `/1000` and has no `OS_SYSTICK_DIV` handling, so a test built with `-DOS_SYSTICK_DIV=2` gets 0.5 ms *logical* ticks (twice the intended sysclock rate) instead of 0.5 ms preemption with a 1 ms sysclock. Confirmed divergence.

[Medium] `OS_HAS_INTERRUPTS_STACK` is declared but the full ISR frame lives on the task stack — `include/cmsis-plus/rtos/port/os-c-decls.h:76` (AArch32) and `:67` (AArch64)

Both ports define `OS_HAS_INTERRUPTS_STACK`, which makes the kernel print and expose a separate interrupt stack, but the AArch32 IRQ handler switches to SVC mode and builds the entire 82-word context frame on the *task's* SVC stack (`src/handlers.cpp:168-192`); only `r0-r3` briefly touch the IRQ stack. The AArch64 linker comment admits the region is "only what μ OS++ records for its bookkeeping" (`aarch64/linker.ld:64-66`). A deep ISR therefore consumes task stack, and the capability advertised to the kernel is not real. Confirmed. Fix: drop the macro, or actually use the interrupt stack for the frame.

[Medium] Pi timer calibration runs on every core with IRQs masked — `test/boards/rpi-zero-2w/src/timer_arm.cpp:50`

```
void init() noexcept {
    g_freq = calibrate_hw_freq();    // busy-waits ~50 ms
    ...
}
```

`port_sys_init()` calls `timer_arm::init()` on every core (`test/boards/rpi-zero-2w/src/port_sys.cpp:33`), and calibration busy-waits 50 ms against the BCM system timer with interrupts masked. That is ~150 ms of extra boot latency and turns a hardware-fragile measurement into four independent measurements of a *shared* counter. Confirmed (perf/robustness). Fix: calibrate once on core 0 and publish the result.

[Medium] AArch32 board-fact contract not enforced in the AArch64 port —

[aarch64/include/cmsis-plus/rtos/port/os-c-decls.h:52](#)

```
#ifndef OS_NCPU
#define OS_NCPU      4
#endif
...
#define OS_SMP_IPI_SGI 0
```

The AArch32 port deliberately `#error`s unless `OS_NCPU` / `OS_SMP_IPI_SGI` come from `board.cmake` ([aarch32 .../os-c-decls.h:58-63](#)), precisely so a board cannot silently get the wrong core count. The AArch64 variant re-introduces silent defaults. Confirmed divergence; the stated contract/logic is weaker on the 64-bit port. Fix: mirror the `#error` contract.

[Low] Deferred publish has no release barrier before the pointer store — [test/](#)

[boards/rpi-zero-2w/src/startup.S:117-131](#) (AArch64) and [aarch32/src/handlers.cpp:199-216](#)

`SMP_PUBLISH` stores `old->stack_ptr` and only then issues `dmb ish`. Frame visibility to a claimant core is actually guaranteed by the kernel lock's release-acquire pairing (the frame is written before `_smp_klock` is released; the claimant acquires the lock before reading the frame), so the current code is correct. But the publish store itself is not ordered as a release, so any future change to when/how the lock is released would silently expose a partially invisible frame. Suspicion/robustness. Fix: use `stlr` (or a `dmb` before the store) and document the dependency.

[Low] AArch32 TTBR0 inner attribute is WB, not WBWA as the comment claims —

[test/boards/rpi-zero-2w/src/mmu.cpp:96](#)

```
uint32_t ttbr = v | (1u << 6) | (1u << 3) | (1u << 1) | (1u << 0);
```

IRGN is encoded as `{bit6, bit0}`; setting both bits selects write-back, not write-allocate, while the comment and the DRAM section mapping use WBWA (`kNormalwbwa`). Both are cacheable+shareable, so coherency and the exclusive monitor still work; this is an attribute inconsistency, not a corruption. Confirmed. Fix: set only bit 6 for IRGN=WBWA.

[Low] AArch64 spin-table release base may be off by one — [test/boards/rpi-zero-2w/](#)

[src/smp.cpp:34](#)

```
return reinterpret_cast<volatile std::uint64_t*>(
    static_cast<std::uintptr_t>(0xD8u + 8u * core));
```

The upstream BCM2837 device tree places CPU1's release word at `0xD8` (formula `0xD8 + 8*(core-1)`), whereas this code writes CPU1 to `0xE0`. The AArch32 shim uses the same `0xD8 +`

`8*core` formula and is reported to work, so either the firmware/QEMU stub here differs from the DT layout or the hardware path never uses these words (tests resume all cores via OpenOCD, `hw.sh` `UOS_HW_RESUME=pc`). Cannot confirm without hardware; flagged as a verification item, not a proven bug.

[Low] AArch32 luckfox linker region overruns the mapped DRAM top — `test/`

`boards/luckfox-lyra/linker.ld:29` VS `src/mmu.cpp:39`

```
RAM (rwx) : ORIGIN = 0x00200000, LENGTH = 0x08000000 /* ends 0x08200000 */
```

```
constexpr uint32_t kDramTop = kDramSizeMb << 20; // 0x08000000
```

The linker believes RAM extends to `0x08200000` while the MMU only maps DRAM below `0x08000000`; anything the linker might place in that 2 MiB tail would translation-fault. Nothing currently lands there, so it is latent. Confirmed inconsistency.

[Low] `udelay` / `get_timer` hard-code 24 MHz, decoupled from `timer_arm` —

`test/boards/luckfox-lyra/usb/src/usb_env_stateos.cpp:36`

```
constexpr std::uint32_t kCntHz = 24000000u; // CNTFRQ set by startup.S
```

`timer_arm::init()` resolves the rate (falling back to 24 MHz only when `CNTFRQ==0`, `test/boards/luckfox-lyra/src/timer_arm.cpp:24-31`), but the USB environment shim hard-codes the same number. They agree today; if `CNTFRQ` ever comes back non-zero-but-different (e.g. a different loader) USB timing silently diverges from the RTOS's. Also `mdelay(ms)` computes `ms * 1000u` in 32-bit `unsigned long`, which overflows for large delays (`usb_env_stateos.cpp:83`). Confirmed.

[Low] AArch32 secondary start has dead code after `reschedule()` — `src/rtos/os-core.cpp:424`

```
os::rtos::port::scheduler::reschedule ();
__asm__ volatile("cpsie i" ::: "memory");
for (;;) { __asm__ volatile("wfi"); }
```

`reschedule()` performs the first `port_ctx_switchHandler()` and never returns, so the interrupt-enable and `wfi` loop are unreachable. Harmless, but it obscures that the secondary depends on the first switch completing. Confirmed (dead code).

[Low] AArch32 `switch_stacks` re-implements `_smp_klock_raw_acquire` inline —

`src/rtos/os-core.cpp:213`

The LDREX/STREX acquisition in `switch_stacks` duplicates the exact instruction sequence of `_smp_klock_raw_acquire()` (`os-inlines.h:70-86`), as does the release logic vs `_smp_klock_raw_release()`. Two copies of a memory-ordering primitive is how the release order regressed before (the comment at `os-core.cpp:302-319` documents a prior SMP freeze from exactly this). Confirmed maintainability/divergence risk; fix: call the helper.

[Low] AArch64 tripwire hard-codes the SPSR offset — `src/rtos/os-core.cpp:249`

```
uint64_t spsr = (p != nullptr) ? *reinterpret_cast<uint64_t*> (p + 248) : 0;
```

`port_ctx.hpp:28` already exports `SPSR_IDX = 248/8` for exactly this purpose, but `os-core.cpp` uses the literal `248` (and the AArch32 tripwire correctly uses `ctx_cpsr_word`). A layout change would silently desynchronise the AArch64 check. Confirmed. Fix: use `SPSR_IDX * sizeof(uint64_t)`.

[Low] Stale/contradictory board comments — `aarch32/test/boards/rpi-zero-2w/src/`

`startup.S:9`, `include/uart.hpp:33`, `aarch64/include/cmsis-plus/os-app-config.h:7`

- `startup.S:9` says "QEMU raspi3b: ALL four cores are started at `_start` simultaneously", but `qemu-raspi3-shim/shim.S` releases cores 1-3 from QEMU's spin-table and `qemu.sh` loads via the shim.
- `uart.hpp:33` says "SEMIHOST is defined only by the hardware-run build", but `test/CMakeLists.txt:66` puts `SEMIHOST` in `_common_defines` for every variant; the same claim is repeated in `include/semihosting.hpp:5`.
- `aarch64/include/cmsis-plus/os-app-config.h:7` still describes "the RK3506 Cortex-A7 SMP port ... OS_NCPU=3" in the AArch64 repo.

All are documentation defects (Nits individually), but the first two can send a bring-up investigation down the wrong path.

[Low] AArch32 timer accessors omit `"memory"` clobbers — `include/`

`timer_arm.hpp:37-65`

`set_freq`/`get_freq`/`set_tval`/`get_ctl` etc. are `__asm__ volatile` MMIO operations with no `"memory"` clobber, so the compiler may move ordinary loads and stores across them. The engine's other register accessors in this tree do add `"memory"`. Confirmed (theoretical reordering); fix: add `"memory"`.

AArch32 vs AArch64 divergences

Beyond the ISA-mandated ones (banked modes/VFP vs PSTATE/Q-registers, CP15 vs system registers, GIC vs the BCM local controller), the following are semantic and worth reconciling:

Topic	AArch32	AArch64
<code>OS_SYSTICK_DIV</code>	supported (<code>period_cycles()</code> , phase counter)	ignored, <code>freq/1000</code> hard-coded
Board-fact contract	<code>#error</code> if <code>OS_NCPU</code> / <code>OS_SMP_IPI_SGI</code> unset	silent defaults (<code>OS_NCPU=4</code> , SGI 0)
Kernel lock asm	<code>LDREX/STREX</code> + <code>dmb</code>	<code>LDAXR/STLXR</code> + <code>dmb ish</code> (fine)
Release barrier	<code>dmb</code> / <code>dsb</code> (full system)	<code>dmb ish</code> / <code>dsb ish</code>
<code>switch_stacks</code> acquire	inline duplicate of helper	uses <code>ldaxr/stlxr</code> inline too
SYS_EXIT parameter	bare <code>int*</code> on hardware	<code>{reason,status}[2]</code> block
Tripwire SPSR offset	<code>ctx_cpsr_word</code> constant	literal <code>248</code>
Fault reporting	rich abort/undef dumps + test-mode resume	minimal ESR/ELR/FAR dump, no resume
<code>_Exit</code> location	separate <code>semihosting-exit.cpp</code>	inside <code>handlers.cpp</code>
<code>input_clock_frequency_hz</code>	uses wrong raw <code>get_freq()</code> (Pi)	raw <code>get_freq()</code> is honest
<code>port_cpu_id</code> mask	<code>&3</code> (port_sys) / <code>&0xFF</code> (inlines)	<code>&0xFF</code>
Interrupt frame stack	task SVC stack; IRQ stack only for r0-r3	task SP_EL1 only

Good practices

- **Deferred publish + claim-by-nulling.** The `_smp_pub_addr/_smp_pub_val` staging, the `stack_ptr = nullptr` claim, and the `th == old_thread || (state != running && stack_ptr != nullptr)` skip in `internal_switch_threads` (`smp/src/rtos/os-core.cpp:600-606`) correctly prevent a second core from running a thread whose stack is still in use.
- **Release ordering documented and implemented.** `switch_stacks` clears `depth` / `owner` before the lock word and explains the prior total-freeze bug (`aarch32/src/rtos/os-core.cpp:302-322` , aarch64 `:278-299`).
- **Tripwires.** Frame-bounds + CPSR/`SPSR` mode validation before every switch (`os-core.cpp:244-287` , aarch64 `:238-268`) turns a protocol violation into an on-the-spot report.
- **Fault console isolation.** `FaultConsole` prints only through `puts_uart` / `putc_uart` , with an excellent rationale for why a fault handler must not touch the semihosting trap (`aarch32/src/exception_handler.cpp:15-37`).
- **Cache maintenance for a caches-off receiver.** `DCCMVAC` / `dc cvac` + `dsb` before `sev` /mailbox release (`aarch32/.../smp.cpp:49-60` , `aarch64/.../smp.cpp:53-59`), with the coherency reasoning in comments.
- **Semihosting trap is treated as a property of the debugger**, selected per board, and the HLT-on-ARMv7 hazard is called out and avoided for the A7 (`include/semihosting.hpp:10-34`).
- **Board-fact centralisation** in `board.cmake` (`OS_NCPU` , `PORT_RAM_*` , `PORT_GREETING`) with `#error` enforcement on the AArch32 side.

Code review — μOS++ Ille Cortex-M and POSIX architecture ports

Scope

Read-only review of two working copies under `/home/dan/Work`:

- `micro-os-plus-iii-cortexm.git`
- the port proper: `include/`, `include-m33/`, `include-rp2350/`, `src/rtos/os-core.cpp`, `src/rtos/os-core-m33.cpp`, `src/rtos/os-core-rp2350.cpp`, `src/semihosting-exit.cpp`, `src/libc/getentropy.c`, `CMakeLists.txt`;
- the board/test trees: `test/boards/pico2/*`, `test/boards/pico2-pizero/*`, `test/boards/pico2-rp2350b-psram/*`, `test/boards/shared/*`, the STM32 boards' glue and the `hw.sh`/`qemu.sh` runners.
- `micro-os-plus-iii-posix-arch.git`
- `include/*`, `include/cmsis-plus/rtos/port/*`, `src/*` (`host_cpu.cpp`, `rtos/os-core.cpp`, `exception_handler.cpp`, `free-store.cpp`, `board-contract.cpp`, `diag/trace-posix.cpp`);
- `test/boards/native/*`, `test/run.sh`.

Sibling repos (`micro-os-plus-iii-smp.git`, `micro-os-plus-iii-devices.git`) were consulted only to establish the kernel-side contracts the ports must honour (`internal_switch_threads()`'s `stack_ptr != nullptr` pick rule, `scheduler::lock()/unlock()` semantics, `exit()` → `_Exit()`). No file was modified except this report.

Summary

The ports are unusually well documented and honest about their design; the SMP model (deferred publish, per-core tick, recursive kernel lock) is coherent and the comments correctly explain the hard parts. The problems below are concentrated in the **divergence between the two Cortex-M SMP cores** (`os-core-m33.cpp` vs `os-core-rp2350.cpp`): the RP2350 copy lost the M33's PRIMASK save/restore, and both copies lost the upstream Cortex-M port's local interrupt masking in `switch_stacks()`. The high-resolution-clock overflow test is wrong (dead code) in all three headers. The POSIX port's most serious portability defect is the macOS tick, which is process- not thread-directed.

No finding here is demonstrated by a failing test in this read-only pass; classifications below separate defects I can prove from the code from suspicions that need a run to trigger.

Findings

[Critical] SMP "no ready thread" path spins forever while holding the kernel spinlock — `src/rtos/os-core-m33.cpp:471`, `src/rtos/os-core-rp2350.cpp:448`

```
if (new_thread == nullptr)
{
    __asm__ volatile ("cpsid if" ::: "memory");
    for (;;)
        __asm__ volatile ("wfi");
}
```

`switch_stacks()` acquired `_smp_klock` immediately above (m33 lines 455–460, rp2350 lines 426–431) and never releases it on this path. If it is ever taken, the other core spins forever in the first critical section it enters — a silent, total SMP freeze rather than a diagnosable abort. It should clear `owner` / `depth` and release the lock (or abort) before spinning. The same defect exists verbatim in both cores.

[High] RP2350 scheduler unlock discards the caller's PRIMASK — `src/rtos/os-core-rp2350.cpp:352`

```
lock_state[cpu] = state;
port_put_lock (0);
```

The M33 core stores the PRIMASK captured when the lock was taken and restores it (`lock_primask[cpu]`, `src/rtos/os-core-m33.cpp:395,408`). The RP2350 core has no such array and passes a hard-coded `0`, so `port_put_lock()` ends with `__set_PRIMASK(0)` and unconditionally re-enables interrupts. The M33 header warns about exactly this (`include-m33/.../os-inlines.h:164-169`: "passing a hardcoded 0 would wrongly re-enable IRQs when the lock is taken with interrupts already disabled"). Any caller that enters the scheduler lock with IRQs masked and relies on them staying masked is silently broken. This is a direct consequence of the copy divergence noted below.

[High] SMP `switch_stacks()` does not mask local interrupts — `src/rtos/os-core-m33.cpp:449`, `src/rtos/os-core-rp2350.cpp:419`

The upstream Cortex-M port disables BASEPRI/PRIMASK for the duration of the picker (`src/rtos/os-core.cpp:842-869`). Both SMP cores instead run the whole context switch with only the cross-core spinlock and with interrupts enabled:

```

stack::element_t*
switch_stacks (stack::element_t* sp)
{
    #if defined(OS_USE_SMP_SCHEDULER)
        unsigned cpu = port_cpu_id ();
        if (_smp_klock.owner != cpu) { _smp_klock_raw_acquire (); ... }
    #endif
}

```

Because PendSV is the lowest-priority exception, every other IRQ can preempt it in the middle of `internal_switch_threads()`'s walk of `ready_threads_list_`. An ISR that calls a kernel API (semaphore post, timer, message queue) enters `critical_section`, and since `port_set_lock()` sees `owner == cpu` it does *not* spin — it recurses and mutates the same ready list the suspended picker is walking. That is exactly the corruption the upstream mask prevents. IRQs should be masked around the switch (or the ISR path must be forbidden, which it is not).

[Medium] High-resolution clock overflow test reads the wrong register — include/cmsis-plus/rtos/port/os-inlines.h:463, include-m33/.../os-inlines.h:397, include-rp2350/.../os-inlines.h:511

```

if (SysTick->CTRL & SCB_ICSR_PENDSTSET_Msk)

```

`SCB_ICSR_PENDSTSET_Msk` is bit 26 of `SCB->ICSR`; bit 26 of `SysTick->CTRL` is reserved and reads 0. The condition is therefore never true and the whole overflow-compensation block (`(load_value + 1 + (load_value - val))`) is dead code. The intended source is almost certainly `SCB->ICSR & SCB_ICSR_PENDSTSET_Msk` (the comment says "If the exception is pending"). Effect: `cycles_since_tick()` can under-report by up to one full period when the tick is pending but not yet serviced, degrading the high-resolution clock. Present identically in all three port headers.

[Medium] The two Cortex-M SMP cores are near-duplicates that have already diverged — src/rtos/os-core-m33.cpp, src/rtos/os-core-rp2350.cpp, include-m33/, include-rp2350/

`CMakeLists.txt:186-194` states the files are "91% the same file, and one day they should be one". They are not merely duplicated: they have diverged in ways that matter. The m33 core carries the `lock_primask[]` fix; the rp2350 core does not (see the High finding above). The m33 core launches core 1 from `start()` via `port_smp_launch_core1()`; the rp2350 core expects the test to call `multicore::launch_core1()` and exposes `port_core1_stack_top()`. The m33 core defines its switch instrumentation globals itself (`os-core-m33.cpp:81-87`); the rp2350 core declares them `extern` and expects the board (`os-core-rp2350.cpp:43-49`). Keeping two copies of a context-switch critical section is how the PRIMASK bug survived; the split should be closed or the shared body factored out.

[Medium] The RP2350 port headers depend on board BSP headers — include-rp2350/.../os-inlines.h:35, src/rtos/os-core-rp2350.cpp:39-40

```
#if defined(OS_USE_SMP_SCHEDULER)
#include <bsp/rp2350.hpp>
#endif
```

```
#include <bsp/multicore.hpp>
#include <bsp/rp2350.hpp>
```

The port's own `src/rtos/` and `include-rp2350/` cannot be compiled without the board's BSP include directory, inverting the stated layering ("cortexm gains SMP by a board arriving that supplies a lock and an IPI, not by this file changing", `CMakeLists.txt:29-34`). Any RP2350 board that names a different BSP header path will not build the port. The SIO/PSM/TICKS register definitions should be provided through a documented port-side interface rather than by direct inclusion of `bsp/`.

[Medium] POSIX macOS tick is process-directed, not per-CPU — src/host_cpu.cpp:220-224

```
#if defined(__linux__)
    sev.sigev_notify = SIGEV_THREAD_ID;
    sev._sigev_un._tid = static_cast<int> (::syscall (SYS_gettid));
#else
    sev.sigev_notify = SIGEV_SIGNAL;
#endif
```

The comment above this block explains precisely why `SIGEV_SIGNAL` is wrong ("delivers SIGALRM to an ARBITRARY thread of the process ... the preemption of CPU 2 could be charged to CPU 0"). On `__APPLE__` (which the file compiles for) the code still takes that branch, so the per-CPU tick model is broken on macOS: all timers target the process, delivery is arbitrary/coalesced, and the "only CPU 0 advances the clock" invariant no longer holds deterministically. Either implement a per-thread mechanism on macOS or gate SMP support on Linux.

[Medium] POSIX fault reporter calls non-async-signal-safe functions — src/exception_handler.cpp:125, :170

```
emit (::strsignal (static_cast<int> (type)));
...
int n = ::backtrace (frames, 24);
```

The file's own contract (lines 32-34) is "uses write(2) and nothing else: no printf, no malloc, no locks." `strsignal()` may take the locale lock / allocate, and `backtrace()` may take the dynamic-loader lock on first use. A deadlock here is unrecoverable — the process is already faulting and the handler is the only diagnostic. `backtrace()` is acknowledged as a calculated risk in the comment; `strsignal()` is not.

[Medium] POSIX `reschedule()` reads the CPU id before masking — `src/rtos/os-core.cpp:412`

```
void reschedule (void)
{
    const unsigned cpu = port_cpu_id ();
```

Every other place that reads the CPU id in this port masks first, and documents why (`src/rtos/os-core.cpp:181-189`, `src/host_cpu.cpp:186-189`, and the comment in `switch_stacks()` at lines 282-300). Here the tick is still enabled, so a signal delivered between `port_cpu_id()` and the later `_port_ctx_pending[cpu] = 1` can switch this context to another host thread; when `reschedule()` resumes, `cpu` names the core it left. It then either marks the wrong CPU pending or tests the wrong `_smp_klock.owner`. This is a suspicion in the sense that the deferral branches are only reached when the lock is held (where callers usually mask), but the code is inconsistent with the port's own rule and should mask before the read.

[Low/Medium] POSIX critical section saves only the tick signal, not the IPI — `src/cmsis-plus/rtos/port/os-inlines.h:197`

```
return ::sigismember (&old, clock::signal_number ()) != 0;
```

The mask is a two-signal set (tick + IPI). `critical_section::exit()` restores with `state ? SIG_BLOCK : SIG_UNBLOCK` over *both* signals (line 216), so a caller that had only the IPI blocked will have it unblocked. The prior state of `ipi_signal_number()` should be captured and restored separately.

[Low/Medium] POSIX per-CPU timers are never deleted — `src/host_cpu.cpp:227`

```
timer_t timer;
if (::timer_create (CLOCK_MONOTONIC, &sev, &timer) != 0) { fatal (...); }
```

`timer_delete()` is never called, so each CPU leaks a POSIX timer for the life of the process. In a long-lived process that starts/stops scheduler instances this accumulates. (Only one scheduler instance exists per process today, so the practical impact is small.)

[Low/Medium] RP2350 IPI push can block inside an interrupt handler — src/rtos/os-core-rp2350.cpp:614, test/boards/pico2/glue/multicore.cpp:60

```
// Non-blocking: if the FIFO is full the target already has a pending IPI.  
void port_smp_ipi (unsigned cpu) { ... multicore::fifo_push (0x4D53u); }
```

```
void fifo_push (std::uint32_t v) { while (!fifo_write_ready ()) { } reg(...) = v; ... }
```

`fifo_write_ready()` is checked in `port_smp_ipi`, but `fifo_push()` re-checks and busy-waits if the FIFO became full in between (the FIFO is shared by both cores). `port_smp_ipi()` is reachable from ISR context; a full FIFO there spins with interrupts masked until the peer drains it. The claim "non-blocking" is not guaranteed by the implementation.

[Low] RP2350 core-1 launch handshake has no timeout — test/boards/pico2/glue/multicore.cpp:126

```
unsigned seq = 0;  
do  
{  
    std::uint32_t cmd = cmd_seq[seq];  
    ...  
    seq = (cmd == response) ? seq + 1 : 0;  
}  
while (seq < 6);
```

The readiness wait above is guarded (2 000 000 iterations), but this echo handshake is not. If core 1 misbehaves (or a FIFO word is lost), the launch livelocks forever. A bounded retry with a diagnostic would fail fast.

[Low] RAM-exec boot stub silently accepts a failed verify — test/boards/pico2/src/boot.S:88

```
stub_retry_bad:  
    subs r8, r8, #1  
    bne stub_retry  
stub_done:  
    movs r0, #0  
    msr primask, r0  
    ldr r0, =_reset_handler
```

After the 32-attempt budget is exhausted the stub falls through into `_reset_handler` and runs a possibly-corrupt copy of the image. On a board whose whole reason for this stub is marginal flash, silently executing mismatched code is the worst outcome; it should signal (LED/loop/semihost) instead.

[Low] `_getentropy()` returns a predictable stream — `src/libc/getentropy.c:28`

```
for (size_t i = 0; i < length; ++i) { p[i] = (uint8_t)(0xA5u ^ (uint8_t)i); }
```

The comment calls it a "Deterministic placeholder", which is honest, but it is named `entropy` and satisfies callers that genuinely need unpredictability (newlib `getentropy`, `arc4random` fallbacks). A test that trusts it can never detect real RNG problems; this should at least be a compile-time opt-in, not the default.

[Low] Fault reporter can deadlock on the UART software lock — `test/boards/pico2/glue/rtos-glue.cpp:116`, `test/boards/pico2/glue/uart.cpp:48`

`fault_report()` writes through `uart::write()`, which takes SIO spinlock 1. If the fault occurred while the faulting core held that spinlock (e.g. inside `uart::write`), the spin in `lock_uart()` never terminates and the beacon loop is never reached. Fault paths generally need a lock-free/raw console write.

[Low] m33 switch-instrumentation globals are owned by the port while rp2350's are owned by the board — `src/rtos/os-core-m33.cpp:81-87`

```
volatile void* g_sw_old;
volatile void* g_sw_new;
...
const char* g_sw_hist[8];
```

`os-core-rp2350.cpp:43-49` declares the same names `extern` and expects the board (`glue/rtos-glue.cpp`) to define them. The m33 core instead defines them itself, non-static and without `extern "C"`. Same names, two different owners, differing linkage — a hazard if the m33 port is ever linked with a board that also defines them.

[Low] `hw_result.hpp` reimplements the semihosting trap — `test/boards/shared/hw_result.hpp:41`

```
register unsigned r0 __asm__ ("r0") = 0x04u; // SYS_WRITE0
register const char* r1 __asm__ ("r1") = s;
__asm__ volatile ("bkpt 0xAB" : : "r" (r0), "r" (r1) : "memory");
```

`src/semihosting-exit.cpp:16-18` argues at length that the kernel's `cmsis-plus/arm/semihosting.h` is the single place that spells the trap. The test helper nonetheless hardcodes `bkpt 0xAB`, so the port now has two sources of truth for the semihosting ABI. It should call the kernel helper.

[Low] POSIX runner delegates to an out-of-tree script — test/boards/native/run.sh:26

```
UOS_RUN_ONLY="${1:-}" \
exec "$SMP_DIR/test_smp1/run-host.sh" "$BUILD/test"
```

The actual per-test execution, timeouts and pass/fail parsing live in the SMP repo and cannot be reviewed here. `$BUILD` is passed through even when it does not exist, so the failure mode is the external script's. The dispatcher (`test/run.sh`) itself is fine.

[Low] POSIX kernel-lock spin has no backoff on architectures other than x86/ARM — include/cmsis-plus/rtos/port/os-inlines.h:104

```
while (__atomic_exchange_n (&smp_klock.lock, 1u, __ATOMIC_ACQUIRE) != 0u)
{
    #if defined(__x86_64__) || defined(__i386__)
        __builtin_ia32_pause ();
    #elif defined(__aarch64__) || defined(__arm__)
        __asm__ volatile("yield" ::: "memory");
    #endif
}
```

On any other host the loop is a tight atomic exchange with no hint — wasteful and potentially unfair to the lock holder under oversubscription, which the README explicitly supports (`OS_NCPU > 16` warning in `src/board-contract.cpp:39`).

[Suspicion] POSIX `switch_stacks()` acquires the kernel lock unconditionally — src/rtos/os-core.cpp:308

```
_smp_klock_raw_acquire ();
_smp_klock.owner = cpu;
_smp_klock.depth = 1;
```

Unlike `critical_section::enter()`, this does not check `owner == cpu` first. It is safe only because both current callers (`reschedule()` and `irq_epilogue()`) return early when this CPU already owns the lock. A future caller reached from inside an `interrupts::critical_section` would self-deadlock on the spin. The recursive-entry guard belongs in `switch_stacks()` itself.

[Suspicion] POSIX high-res fallback shares `previous_timestamp` without synchronization — `src/rtos/os-core.cpp:561`

`static uint64_t previous_timestamp;` is plain storage read/written by `cycles_per_tick()` and `cycles_since_tick()`. Today only CPU 0 calls `cycles_per_tick()` (inside `os_systick_handler()`), and `has_hardware_counter()` returns true on this port so `hrclock::now()` uses `hardware_counter()` instead. If either assumption changes (a port where the fallback is live, or a caller on a non-zero CPU), this becomes an unsynchronized cross-CPU read-modify-write. It should be per-CPU or atomic.

[Suspicion] POSIX load-bias subtraction is meaningful only for this image — `src/exception_handler.cpp:131`

```
emit_hex (elr - g_load_bias);
```

`g_load_bias` is obtained from `dldaddr(&port_fatal_exception)`, i.e. the load base of the executable. Subtracting it from a return address that belongs to a shared object (libc, libstdc++) yields a meaningless "static" address. The `backtrace()` lines at 175 have the same problem. It is only cosmetic in a crash dump, but the label "static" overstates its accuracy.

Good practices

- The comments are exceptional: the SMP handover ordering, the deferred-publish rule, and the linker-order requirement for releasing owner/depth before the lock word (`src/rtos/os-core.cpp:360-373`) are documented with the failure mode they prevent, which made this review far faster.
- The Cortex-M generic core and the POSIX port both keep the `stack_ptr != nullptr` pick rule and the "mask before reading the CPU id" rule in one place, and cross-reference the AArch64/AArch32 equivalents.
- `src/semihosting-exit.cpp` correctly routes through the kernel's single trap definition, and `multicore.cpp:25-34` explicitly documents why `(void)reg(...)` is *not* enough to pop the SIO FIFO — a subtle, real C++ trap avoided.
- `psram.cpp`, `clocks.cpp` and `multicore.cpp` bound their hardware waits where it matters (PSRAM QMI, PLL lock, bootrom readiness), and the PSRAM init masks IRQs around the direct-mode window with an explicit rationale.
- The POSIX port uses `_exit()` in `hw_result.hpp` (with a `fflush`) deliberately to avoid running static destructors underneath live CPUs — the right call.
- The RP2350 reset handler releases all 32 SIO spinlocks (`test/boards/pico2/src/boot.S:127-135`) so a debugger reset cannot leave the next image spinning on a lock held by the previous one.
- `board.cmake` / `tests.cmake` discovery, the `OS_NCPU`-derived `OS_USE_SMP_SCHEDULER`, and the two-half board/ISA interface split are clean and make per-board differences auditable.

Read-only code review — μOS++ IIIe test harness / platforms / build system

Scope

Reviewed, read-only, under `/home/dan/Work/micro-os-plus-iii-smp.git`:

- `tests/CMakeLists.txt`, `tests/cmake/{global-definitions,common-options,tests-main}.cmake`
- `tests/platforms/*/` — every platform's `CMakeLists.txt`, `cmake/{definitions,platform-library,dependencies-folders}.cmake`, `src/platform-support.cpp` (5 copies), `include/`
- `tests/sources/*/` — the portable suites' CMake and sources (read for robustness)
- `tests/device-qemu-cortexm/CMakeLists.txt`
- `tests/package.json` (build configurations/actions)
- Harness docs: `tests/README.md`, `docs/tests/README.md`, `docs/tests/README-DEVELOPER.md`, `docs/tests/README-MAINTAINER.md`, and (for cross-checks) `docs/tests/HARNESS-TESTS-PARADIGM.md`, `docs/tests/HARNESS-BOARD-TEST-CHEATSHEET.md`, `docs/tests/TESTS-CATALOG.md`
- Shell helpers invoked by the harness: `test_smpl/{run-qemu,run-hw,run-host}.sh`, plus the deprecated `tests/deprecated/scripts/*.sh`
- The port-owned builders/boards that the harness `add_subdirectory()`s were consulted only as evidence of the harness contract (they are separate repos).

No source files were modified. Existing build trees under `tests/build/` were inspected read-only; two existing QEMU images were re-run to test a suspicion (no tree change).

Summary

The harness is well-commented and the "port owns its tests, platform registers what the builder produced" seam is sound; the newer `cortexm-*`, `native`, `aarch32-*`, `aarch64-*` platforms follow it consistently. The most serious issues are (1) a real functional divergence in the Luckfox platform-support file that never releases the board's secondary cores, (2) a large amount of dead/misleading `platform/platform-support` CMake on the four Raspberry Pi platforms, hard-wired to the QEMU variant, and (3) build-hygiene gaps (toolchain guards only on aarch32/aarch64, an `ENABLE_HW_TESTS` flag that nothing reads, hard-coded sibling-repo paths). Documentation is partly acknowledged as historical, but `tests/README.md` and the two upstream READMEs are genuinely stale.

Two suspected defects were investigated and **disproved**; they are not listed: the `${rpath_options_list}` generator-expression is handled correctly by CMake (both `-wl, -rpath` and `-L` appear), and the recorded `2xcortex-m33-fp-switch-test` failure reproduces as PASS on re-run.

Findings

[High] Luckfox harness never releases the board's secondary cores — `tests/platforms/aarch32-luckfox-lyra/src/platform-support.cpp:94`

The Luckfox Lyra board is a 3-core SMP Cortex-A7 (`micro-os-plus-iii-aarch32.git/test/boards/luckfox-lyra/board.cmake:14` → `UOS_BOARD_NCPU 3`), and the platform sets `OS_USE_SMP_SCHEDULER=1` (`tests/platforms/aarch32-luckfox-lyra/cmake/platform-library.cmake:53`). Its harness trampoline, however, only installs the idle thread via `main()` and never brings up cores 1..2:

```
[[noreturn]] static void
harness_main_trampoline (void)
{
    int argc = 0;
    char** argv = nullptr;
    os_startup_initialize_args (&argc, &argv);
    int code = os_main (argc, argv);
    ...
}
```

The sibling AArch32 support files do it explicitly (`tests/platforms/aarch32-rpi-zero-2w/src/platform-support.cpp:106`):

```
smp_install_boot_threads ();
smp::start_secondary_cores ();
extern int test_wait_secondaries (int timeout_ms);
test_wait_secondaries (3000);
```

Confirmed in the configured build: the `mutex-stress-test-hwd` link line contains `.../luckfox-lyra/src/smp.cpp.obj` (which defines `start_secondary_cores`) and `platform-support.cpp.obj`, but nothing in the tests/ trampoline calls it (`tests/build/aarch32-luckfox-lyra-cmake-gcc-debug/build.ninja:705`). Consequence: on the Lyra the harness suite runs on core 0 only, and the suite's SMP assumptions (idle threads per core / migration) are never exercised. This is a divergence from the AArch32 rpi support that should be shared, not a local stylistic choice. Severity High (functional test-coverage hole on the only hardware-only AArch32 board); confirmed.

[High] `platform-support` / `base platform` are dead code on the four Raspberry Pi platforms — `tests/platforms/aarch32-rpi3b/cmake/platform-library.cmake:102`

`platform-library.cmake` builds `micro-os-plus::platform` and `micro-os-plus::platform-support` for `aarch32-rpi-zero-2w`, `aarch32-rpi3b`, `aarch64-rpi-zero-2w`, `aarch64-rpi3b`. None of them is referenced anywhere: the port's own builder compiles `src/platform-support.cpp` directly (`micro-os-plus-iii-aarch32.git/test/rpi3b/tests.cmake:92`) and links the port, not these targets. Confirmed:

`micro-os-plus::platform-support` / `platform-aarch32-rpi-zero-2w-support-interface` do not appear in `tests/build/aarch32-rpi-zero-2w-cmake-gcc-debug/build.ninja` at all. The ~120 lines per platform are unreachable and, worse, misleading (see next finding). Low direct risk, but the misleading "adds the startup hooks ... for the harness suites" comment (`:96`) is contradicted by the code path that actually builds the suites.

[Medium] Raspberry Pi base platform hard-codes the QEMU variant — `tests/`

`platforms/aarch32-rpi3b/cmake/platform-library.cmake:54`

Even though these platforms register both `-qemu` and `-hwd` tests, the base interface unconditionally defines `QEMU_BUILD` and selects the QEMU linker:

```
QEMU_BUILD
...
target_link_options (
  platform-aarch32-rpi3b-interface
  INTERFACE -nostartfiles -Wl,--gc-sections "-T${UOS_BOARD_LINKER_QEMU}"
)
```

and `platform-support` inherits it (`:112-120`). Today the hwd path avoids this only because the port's builder is used instead (previous finding). If the dead interface is ever revived for a hardware image, it will silently select the emulator's SVC/reason-by-value semihosting and the QEMU linker script — exactly the trap the Luckfox file warns about in `tests/platforms/aarch32-luckfox-lyra/cmake/platform-library.cmake:16` . Same pattern in `aarch64-rpi3b:53,69` and both `rpi-zero-2w` files. Confirmed code; latent today.

[Medium] Pinned-toolchain guard exists only on aarch32/aarch64 — `tests/`

`platforms/aarch32-rpi3b/cmake/definitions.cmake:34`

The guard that refuses a non-15.2 compiler is present only in the aarch32/aarch64 definitions:

```
if (DEFINED CMAKE_C_COMPILER_VERSION
    AND NOT CMAKE_C_COMPILER_VERSION MATCHES "^15\\.2\\.")
  message (FATAL_ERROR "This platform must be built with the pinned xPack arm-none-eabi-gcc
  15.2 ...")
```

`qemu-cortex-*`, `cortexm-*`, `raspberrypi-pico`, `nucleo-*` and all `native-*` configs have no equivalent, even though they also pin tools in `package.json` (`tests/package.json:551` arm-none-eabi-gcc 15.2.1, `:659` qemu 8.2.6, gcc/clang versions). Combined with the fact that `commandCMakeReconfigure` omits `CMAKE_TOOLCHAIN_FILE` (it is only added by `commandCMakePrepareWithToolchain`, `tests/package.json:50-51`), a bare `cmake` or a build tree whose `xpack/` was removed silently falls back to `/usr/bin/cc` (the doc itself names this failure: `docs/tests/HARNESS-BOARD-TEST-CHEATSHEET.md:583`). Confirmed inconsistency; Medium.

[Medium] Legacy `else` fallback pairs the local SMP kernel with the released Cortex-M port — `tests/cmake/tests-main.cmake:106`

```
else ()
    # Fallback: the plain kernel, one level above (upstream behaviour).
    add_subdirectory (".." "top-bin")
endif ()
```

This branch is taken by `qemu-cortex-*`, `raspberrypi-pico` and `nucleo-*`. Their `dependencies-folders.cmake` then also add the *published* port, e.g. `tests/platforms/qemu-cortex-m4f/cmake/dependencies-folders.cmake:30`:

```
"${CMAKE_BINARY_DIR}/xpacks/@micro-os-plus/micro-os-plus-iii-cortexm"
```

i.e. `@micro-os-plus/micro-os-plus-iii-cortexm@1.1.0` from `package.json:525`, while the kernel under test is the *local* `..` (this SMP repo). The newer `cortexm-*` platforms instead `add_subdirectory` `${UOS_CORTEXM_DIR}` (the local port). So the same harness tests two different kernel/port pairings depending on platform; the qemu/nucleo/raspberrypi ones test the local kernel against a released port, which is a stale combination and a reproducibility hazard. Confirmed; Medium.

[Medium] `ENABLE_HW_TESTS=ON` is passed but no CMake reads it — `tests/package.json:1725`

```
"commandCMakeReconfigure": "cmake ... -D PLATFORM_NAME={{ properties.platformName }} -D
ENABLE_HW_TESTS=ON"
```

`rg ENABLE_HW_TESTS` finds it only in docs (`docs/tests/HARNESS-BOARD-TEST-CHEATSHEET.md:312`, `docs/tests/WORK-SMP-AARCH32-AARCH64-HARNESS-GUIDE.md:911`) and this package.json. The actual `tests/platforms/aarch32-luckfox-lyra/CMakeLists.txt` neither declares the option nor branches on it; it always registers every `hwd` case. So the flag is a no-op and the documented `option(ENABLE_HW_TESTS ...)` does not exist. Confirmed; Medium (dead/drifted configuration).

[Medium] AArch64 support stubs `_gettimeofday` to a constant — `tests/platforms/aarch64-rpi3b/src/platform-support.cpp:161`

```
int
_gettimeofday (struct timeval* tv, void*)
{
    if (tv != nullptr) { tv->tv_sec = 0; tv->tv_usec = 0; }
    return 0;
}
```

The suites exercise the realtime clock heavily (`tests/sources/rtos-apis/src/test-iso-api.cpp:461-511`, `sleep_for<realtime_clock>`, `wait_until(realtime_clock::now()+...)`). A constant epoch makes elapsed-time deltas zero, so those assertions can pass (or misbehave) for the wrong reason. The comment justifies the stub for mutex-stress, but on AArch64 `__ARM_EABI__` is defined, so mutex-stress takes the `hrclock` branch, not `gettimeofday`. Suspicion (Medium) — I did not run an AArch64 image to prove a false pass; worth a targeted check.

[Medium] `bash`-only CTest commands undermine native-Windows support —

`tests/platforms/native/CMakeLists.txt:52`

```
COMMAND bash "${_runner}/run-host.sh" "${_port_bin}"
```

The new harness registers `bash run-*.sh` for native, qemu and hwd everywhere (`tests/platforms/aarch32-rpi-zero-2w/CMakeLists.txt:73,87`, `tests/platforms/cortexm-pico2/CMakeLists.txt:86`, ...). The legacy ports at least tried a Windows shim (`tests/cmake/tests-main.cmake:35-37` sets `extension = .cmd` and legacy commands use `qemu-system-arm${extension} / openocd${extension}`). `tests/README.md:39` claims the tests run on Windows. Unless a Git-Bash `bash` is guaranteed on `PATH`, the entire new harness is non-runnable on native Windows; and `ENVIRONMENT "...;..."` with semicolon-separated absolute Windows paths (`aarch32-rpi-zero-2w/CMakeLists.txt:79-80`) is fragile for paths with spaces. Confirmed code; portability suspicion (Medium).

[Medium] Freshness/labels/timeout missing on the legacy QEMU platforms —

`tests/platforms/qemu-cortex-m4f/CMakeLists.txt:61`

```
add_test (
  NAME "rtos-apis-test"
  COMMAND qemu-system-arm${extension} --machine mps2-an386 ...
)
```

Unlike the newer platforms, these tests carry no `LABELS`, no `TIMEOUT`, and names without the `_${PLATFORM_NAME}-` prefix (the project's own rule, `docs/tests/HARNESS-BOARD-TEST-CHEATSHEET.md:567`). A hung emulator therefore burns CTest's 1500 s default, and there is no `qemu` label to filter. Same for `qemu-cortex-m0/m3/m7f` and the legacy `nucleo-*` / `raspberrypi-pico` names. Confirmed; Medium.

[Medium] Two Cortex-M SMP-QEMU platforms run QEMU with no log capture —

`tests/platforms/2xcortex-m33/CMakeLists.txt:37`

```
add_test (
  NAME "${PLATFORM_NAME}-${name}"
  COMMAND "${_qemu}" --machine ${_qemu_machine} --cpu ${_qemu_cpu} --smp 2 ...
)
```

`2xcortex-m33` and `pico2-1cpu` (and the `cortexm-pico2` qemu cases) invoke QEMU directly, whereas the AArch ports go through `test_smpl/run-qemu.sh`, which tees each run to `.qemu-logs/<app>.log` (`test_smpl/run-qemu.sh:95,118`). `tests/build/2xcortex-m33-cmake-gcc-debug/Testing/Temporary/LastTestsFailed.log` records `2xcortex-m33-fp-switch-test` failing at some point; a re-run of the exact existing image passed (79 migrations, 0 bad), but because no log was kept the original failure cannot be diagnosed. Robustness/diagnostics gap; Medium.

[Medium] Hard-coded sibling-repo paths with a `.git` suffix — `tests/cmake/tests-`

`main.cmake:62`

```
set (UOS_AARCH32_DIR "${CMAKE_SOURCE_DIR}/../../micro-os-plus-iii-aarch32.git"
    CACHE PATH "µOS++ III AArch32 port working copy")
```

The harness only works if the sibling checkouts are literally named `*.git`; the project's own cheat-sheet documents the workaround of symlinking them (`docs/tests/HARNESS-BOARD-TEST-CHEATSHEET.md:623-625`). The values are `CACHE PATH`, so a stale `CMakeCache.txt` keeps pointing at a previous path even after the tree moves. Confirmed; Medium (build-config hygiene).

[Medium] Dead validation messages point at the wrong file — `tests/device-qemu-`

`cortexm/CMakeLists.txt:44`

```
message (FATAL_ERROR
    "Define xpack_device_linker_script_file_name in platforms/${PLATFORM_NAME}/cmake/dependencies-
    folders.cmake")
```

The variable is actually set in `cmake/definitions.cmake` (e.g. `tests/platforms/qemu-cortex-m4f/cmake/definitions.cmake:29`), as is `xpack_platform_compile_definition` referenced by `tests/platforms/qemu-cortex-m4f/cmake/platform-library.cmake:30`. This message is reproduced in every generated platform library. Low severity but actively misleading during bring-up. Confirmed.

[Low] `run-hw.sh` passes a user-controlled string to `printf` as format — `test_smpl/`

`run-hw.sh:93`

```
target_name () { printf "$TARGET_FMT" "$1"; }
```

`UOS_HW_TARGET_FMT` comes from the board (default `bcm2837.cpu%d`). ShellCheck flags SC2059; a stray `%` in a future board config produces garbage or a printf error rather than a clean failure. Also the whole OpenOCD command is assembled by string interpolation of `UOS_HW_*` into a `.cfg` (`:203-216`), which is fine for trusted board files but is a quoting/injection foot-gun. Low; Nit.

[Low] `native` runner scan and exit-on-first-non-executable — `test_smpl/run-`

`host.sh:57`

```
for exe in "${BUILD_DIR}"/*-host; do
  [[ -x "$exe" ]] || { echo "no *-host executables in ${BUILD_DIR}"; exit 2; }
```

Each of the ~17 `native-<app>-host` CTest cases runs `run-host.sh`, which scans the whole directory and filters by `UOS_RUN_ONLY` (`tests/platforms/native/CMakeLists.txt:50-57`). Besides the $O(n^2)$ work, the `-x` check runs before the filter, so a single non-executable `*-host` entry makes every case exit 2. Low.

[Low] Legacy QEMU README errors — `tests/platforms/qemu-cortex-m0/README.md:35`

```
qemu-system-arm --machine mps2-an365 --cpu cortex-m3 ...
qemu-system-arm --machine mps2-an5365 ...
```

`an365` / `an5365` are typos for `an385` (the CMake uses `mps2-an385`, `tests/platforms/qemu-cortex-m0/CMakeLists.txt:68`). `tests/platforms/qemu-cortex-m3/README.md:1` is titled `# platforms/qemu-cortex-m0`. The M0-on-M3 CPU choice itself is intentional and documented in `docs/tests/TESTS-CATALOG.md:65`, so only the typos/title are defects. Low.

[Low] `TRACE` / `-v` leak from the shared interface — `tests/cmake/common-options.cmake:32`

```
$$<CONFIG:Debug>:DEBUG>
$$<CONFIG:Debug>:TRACE>
OS_USE_OS_APP_CONFIG_H
```

`micro-os-plus::common-options` is linked PRIVATE by every executable, so a compiled-in `TRACE` and the linker `-v` (`:64`) apply to *all* sources of every suite, including suites that did not opt in. The old `mutex-stress` header warns that trace in the scheduler interrupt "occasionally" breaks tests (`tests/README.md:74-75`). Intentional for Debug perhaps, but the interface name promises "common" while injecting a debug-only diagnostic macro. Suspicion/Low.

[Low] `EXCLUDE_FROM_ALL` on INTERFACE libraries is meaningless —

`tests/sources/rtos-apis/CMakeLists.txt:30`

```
add_library (test-rtos-apis-interface INTERFACE EXCLUDE_FROM_ALL)
```

`EXCLUDE_FROM_ALL` has no effect on INTERFACE libraries (they produce no build artifacts). Same in every `sources/*/CMakeLists.txt` and `tests/cmake/common-options.cmake:25`. Nit — but it suggests the pattern was copied from object libraries.

[Low] `ENABLE_*` globals only affect legacy platforms — `tests/cmake/global-definitions.cmake:16`

```
set (ENABLE_RTOS_APIS_TEST true)
set (ENABLE_MUTEX_STRESS_TEST true)
set (ENABLE_CMSIS_OS_VALIDATOR_TEST true)
```

Only `qemu-cortex-*`, `nucleo-*` and `raspberrypi-pico` consult these. `native` builds `cmsis-os-validator-test` unconditionally and ignores the flags (`tests/platforms/native/CMakeLists.txt:70`), and the `aarch32/aarch64/cortexm` platforms build everything the port's builder emits. Low (dead configuration surface).

[Low] `package.json` link/install hygiene — `tests/package.json:157`

`link-deps-all` repeats `native-cmake-gcc13-debug` and omits a `gcc14-release` pairing:

```
"xpm run link-deps --config native-cmake-gcc13-debug",
"xpm run link-deps --config native-cmake-gcc13-release",
"xpm run link-deps --config native-cmake-gcc13-debug",
"xpm run link-deps --config native-cmake-gcc14-release",
```

Separately, the native dependency key is `@micro-os-plus/posix-arch` (`:509`) but the link action uses `@micro-os-plus/micro-os-plus-iii-posix-arch` (`:489`); `xpm` happens to install under the package's real name (`tests/build/native-cmake-gcc-debug/xpacks/@micro-os-plus/micro-os-plus-iii-posix-arch`), so the key is misleading. Low.

[Low] Platform-support is quadruplicated and already divergent — `tests/platforms/aarch32-rpi3b/src/platform-support.cpp:1`

The two AArch32 files differ only in comments (verified by `diff`), and the two AArch64 files likewise; AArch32 vs AArch64 differ in the C-library policy. The root repo already offers `micro-os-plus::test-support` for shared scaffolding, but it only carries `hw_result.hpp`/`board-contract.cpp` (`CMakeLists.txt:226-230`), so the ~150-line startup/`main`/console-mirror body is copied four times — which is precisely how the Luckfox file lost its SMP bring-up. Maintainability; Low. Suggest a shared header/target so the divergence cannot recur.

[Low] Stale `tests/README.md` — `tests/README.md:56`

```
For Cortex-M tests, the toolchain is arm-none-eabi-gcc 14.
```

The pin is 15.2.1 (`tests/package.json:551`). The file also references `micro-os-plus-iii.git` throughout (`:64,69`), `.github/workflows/{ci,test-all}.yml` that do not exist in this repo, and omits the newer platforms (`aarch32-*`, `aarch64-*`, `cortexm-pico2`, `2xcortex-m33`, `pico2-1cpu`,

`cortexm-*`). Unlike the `docs/tests/` harness documents it has **no** "earlier layout — read as history" banner, so it reads as current. Medium-adjacent, but doc-only → Low.

[Low] Developer/maintainer READMEs point at the wrong repo/version — `docs/`

`tests/README-DEVELOPER.md:53`

```
xpm run install-native-cmake-sys -C ~/Work/micro-os-plus-iii/micro-os-plus-iii.git/tests
xpm run install-native-cmake-sys -C ~/Work/micro-os-plus-iii/micro-os-plus-iii.git/tests
```

The second command is a copy-paste of the first (should be `test-native-cmake-sys` or `test`). Every path uses the old `micro-os-plus-iii.git` repo. `docs/tests/README-MAINTAINER.md:86-93,116` references a `CHANGELOG.md` and version `7.1.0` that do not exist here (`tests/package.json` is `0.0.0`; no `CHANGELOG.md`). Low.

[Low] Naming rule violated by legacy platforms — `tests/platforms/qemu-cortex-m0/`

`CMakeLists.txt:65`

The docs state the rule "Unique test names + project name per platform (`${PLATFORM_NAME}-...`)" (`docs/tests/HARNESS-BOARD-TEST-CHEATSHEET.md:567`), but `qemu-cortex-*` register bare `rtos-apis-test`/`mutex-stress-test`/`cmsis-os-validator-test`, and `raspberrypi-pico`/`nucleo-*` do the same. Collisions are avoided only because each configuration is a separate build tree; a combined `ctest` across trees would collide. Low.

Documentation drift

- `tests/README.md` — genuinely stale (no history banner): pins "arm-none-eabi-gcc 14" vs `package.json:551` = 15.2.1; references `micro-os-plus-iii.git` and nonexistent `.github/workflows/*`; platform/toolchain list predates aarch32/aarch64/cortexm-pico2/2xcortex/pico2-1cpu.
- `docs/tests/README-DEVELOPER.md` — duplicated `install-native-cmake-sys` (`:51,53`), all paths under the old repo name.
- `docs/tests/README-MAINTAINER.md` — references `CHANGELOG.md` and release `7.1.0` that do not exist here; old repo path.
- `docs/tests/HARNESS-BOARD-TEST-CHEATSHEET.md` / `HARNESS-TESTS-PARADIGM.md` — carry an explicit "earlier layout — read as history" banner at the top, so the wrong `micro-os-plus-iii.git` paths and the sample ctest output `aarch32-rpi-zero-2w-mutex-stress-test` (actual names are `...-mutex-stress-qemu`/`-hwd`, `tests/platforms/aarch32-rpi-zero-2w/CMakeLists.txt:71,86`) are self-declared historical, **except**: `HARNESS-TESTS-PARADIGM.md:228` still recommends `xpm run test-mutex-stress --config ...`, an action that does not exist (package.json only defines `test-mutex-stress-qemu`/`-hwd`), and the "which library is under test" table (`:176-181`) omits the `native`/`cortexm`/`pico2`/`2xcortex` branches that `tests-main.cmake:84-105` now has. Consider moving the current truth into `docs/tests/STEPS.md`/`TESTS-CATALOG.md` (which are current) and deleting or clearly archiving the rest.

- `ENABLE_HW_TESTS` — documented as an option in the cheat-sheet/guide but nonexistent in the CMake (see finding).

Good practices

- The "platform registers what the port's builder produced by `file(GLOB)` -ing `test/<board>/*`" seam means a test added to a port appears with no harness edit (`tests/platforms/aarch32-rpi3b/CMakeLists.txt:63-110`), and the tests-main comment at `:55-60` explicitly documents the duplicate-`micro-os-plus::iii` -alias hazard.
- `run-qemu.sh` / `run-host.sh` / `run-hw.sh` share a single verdict protocol (`RESULT: PASS/SKIP/FAIL` , `TIMEOUT` , `NO-RESULT`) and per-test wall-clock tables, with clear separation of the emulator and the probe; `run-hw.sh` correctly refuses to run a suite (`:280-286`) and reports a lost DAP distinctly from a firmware fault.
- Platform-support mirrors the port's own startup hooks exactly and explains why `putc_uart()` rather than `putc()` is used for the UART mirror (`tests/platforms/aarch32-rpi3b/src/platform-support.cpp:176-179`).
- The `aarch32-luckfox-lyra` platform deliberately omits `QEMU_BUILD` and documents why selecting the emulator trap under OpenOCD would report the wrong status — a good example of encoding a non-obvious board fact next to the code.
- `tests/cmake/tests-main.cmake:35-37` centralizes the Windows command-extension shim, and `package.json` defines per-config tool dependencies rather than relying on the host.